



OpenShift Container Platform 4.14

Installing an on-premise cluster with the Agent-based Installer

Installing an on-premise OpenShift Container Platform cluster with the Agent-based
Installer

OpenShift Container Platform 4.14 Installing an on-premise cluster with the Agent-based Installer

Installing an on-premise OpenShift Container Platform cluster with the Agent-based Installer

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document describes how to install an on-premise OpenShift Container Platform cluster with the Agent-based Installer.

Table of Contents

CHAPTER 1. PREPARING TO INSTALL WITH THE AGENT-BASED INSTALLER	4
1.1. ABOUT THE AGENT-BASED INSTALLER	4
1.2. UNDERSTANDING AGENT-BASED INSTALLER	4
1.2.1. Agent-based Installer workflow	5
1.2.2. Recommended resources for topologies	6
1.3. ABOUT FIPS COMPLIANCE	7
1.4. CONFIGURING FIPS THROUGH THE AGENT-BASED INSTALLER	7
1.5. HOST CONFIGURATION	8
1.5.1. Host roles	8
1.5.2. About root device hints	9
1.6. ABOUT NETWORKING	10
1.6.1. DHCP	10
1.6.2. Static networking	10
1.7. REQUIREMENTS FOR A CLUSTER USING THE PLATFORM "NONE" OPTION	12
1.7.1. Platform "none" DNS requirements	13
1.7.1.1. Example DNS configuration for platform "none" clusters	15
1.7.2. Platform "none" Load balancing requirements	17
1.7.2.1. Example load balancer configuration for platform "none" clusters	18
1.8. EXAMPLE: BONDS AND VLAN INTERFACE NODE NETWORK CONFIGURATION	20
1.9. EXAMPLE: BONDS AND SR-IOV DUAL-NIC NODE NETWORK CONFIGURATION	22
1.10. SAMPLE INSTALL-CONFIG.YAML FILE FOR BARE METAL	24
1.11. VALIDATION CHECKS BEFORE AGENT ISO CREATION	27
1.11.1. ZTP manifests	27
1.12. NEXT STEPS	28
CHAPTER 2. UNDERSTANDING DISCONNECTED INSTALLATION MIRRORING	29
2.1. MIRRORING IMAGES FOR A DISCONNECTED INSTALLATION THROUGH THE AGENT-BASED INSTALLER	29
2.2. ABOUT MIRRORING THE OPENSIFT CONTAINER PLATFORM IMAGE REPOSITORY FOR A DISCONNECTED REGISTRY	29
2.2.1. Configuring the Agent-based Installer to use mirrored images	30
CHAPTER 3. INSTALLING A CLUSTER	32
3.1. PREREQUISITES	32
3.2. INSTALLING OPENSIFT CONTAINER PLATFORM WITH THE AGENT-BASED INSTALLER	32
3.2.1. Downloading the Agent-based Installer	32
3.2.2. Creating the configuration inputs	32
3.2.3. Creating and booting the agent image	35
3.2.4. Verifying that the current installation host can pull release images	35
3.2.5. Tracking and verifying installation progress	37
3.3. GATHERING LOG DATA FROM A FAILED AGENT-BASED INSTALLATION	38
CHAPTER 4. INSTALLING A CLUSTER WITH CUSTOMIZATIONS	40
4.1. PREREQUISITES	40
4.2. INSTALLING OPENSIFT CONTAINER PLATFORM WITH THE AGENT-BASED INSTALLER	40
4.2.1. Downloading the Agent-based Installer	40
4.2.2. Creating the preferred configuration inputs	40
4.2.3. Creating additional manifest files	44
4.2.3.1. Creating a directory to contain additional manifests	44
4.2.3.2. Disk partitioning	44
4.2.4. Using ZTP manifests	46
4.2.5. Encrypting the disk	47

4.2.6. Creating and booting the agent image	48
4.2.7. Verifying that the current installation host can pull release images	48
4.2.8. Tracking and verifying installation progress	50
4.3. SAMPLE GITOPS ZTP CUSTOM RESOURCES	52
4.4. GATHERING LOG DATA FROM A FAILED AGENT-BASED INSTALLATION	55
CHAPTER 5. PREPARING PXE ASSETS FOR OPENSIFT CONTAINER PLATFORM	57
5.1. PREREQUISITES	57
5.2. DOWNLOADING THE AGENT-BASED INSTALLER	57
5.3. CREATING THE PREFERRED CONFIGURATION INPUTS	57
5.4. CREATING THE PXE ASSETS	61
CHAPTER 6. PREPARING AN AGENT-BASED INSTALLED CLUSTER FOR THE MULTICLUSTER ENGINE FOR KUBERNETES OPERATOR	63
6.1. PREREQUISITES	63
6.2. PREPARING AN AGENT-BASED CLUSTER DEPLOYMENT FOR THE MULTICLUSTER ENGINE FOR KUBERNETES OPERATOR WHILE DISCONNECTED	63
6.3. PREPARING AN AGENT-BASED CLUSTER DEPLOYMENT FOR THE MULTICLUSTER ENGINE FOR KUBERNETES OPERATOR WHILE CONNECTED	65
CHAPTER 7. INSTALLATION CONFIGURATION PARAMETERS FOR THE AGENT-BASED INSTALLER ...	71
7.1. AVAILABLE INSTALLATION CONFIGURATION PARAMETERS	71
7.1.1. Required configuration parameters	71
7.1.2. Network configuration parameters	72
7.1.3. Optional configuration parameters	75
7.2. AVAILABLE AGENT CONFIGURATION PARAMETERS	80
7.2.1. Required configuration parameters	80
7.2.2. Optional configuration parameters	81

CHAPTER 1. PREPARING TO INSTALL WITH THE AGENT-BASED INSTALLER

1.1. ABOUT THE AGENT-BASED INSTALLER

The Agent-based installation method provides the flexibility to boot your on-premise servers in any way that you choose. It combines the ease of use of the Assisted Installation service with the ability to run offline, including in air-gapped environments. Agent-based installation is a subcommand of the OpenShift Container Platform installer. It generates a bootable ISO image containing all of the information required to deploy an OpenShift Container Platform cluster, with an available release image.

The configuration is in the same format as for the installer-provisioned infrastructure and user-provisioned infrastructure installation methods. The Agent-based Installer can also optionally generate or accept Zero Touch Provisioning (ZTP) custom resources. ZTP allows you to provision new edge sites with declarative configurations of bare-metal equipment.

1.2. UNDERSTANDING AGENT-BASED INSTALLER

As an OpenShift Container Platform user, you can leverage the advantages of the Assisted Installer hosted service in disconnected environments.

The Agent-based installation comprises a bootable ISO that contains the Assisted discovery agent and the Assisted Service. Both are required to perform the cluster installation, but the latter runs on only one of the hosts.

The **openshift-install agent create image** subcommand generates an ephemeral ISO based on the inputs that you provide. You can choose to provide inputs through the following manifests:

Preferred:

- **install-config.yaml**
- **agent-config.yaml**

or

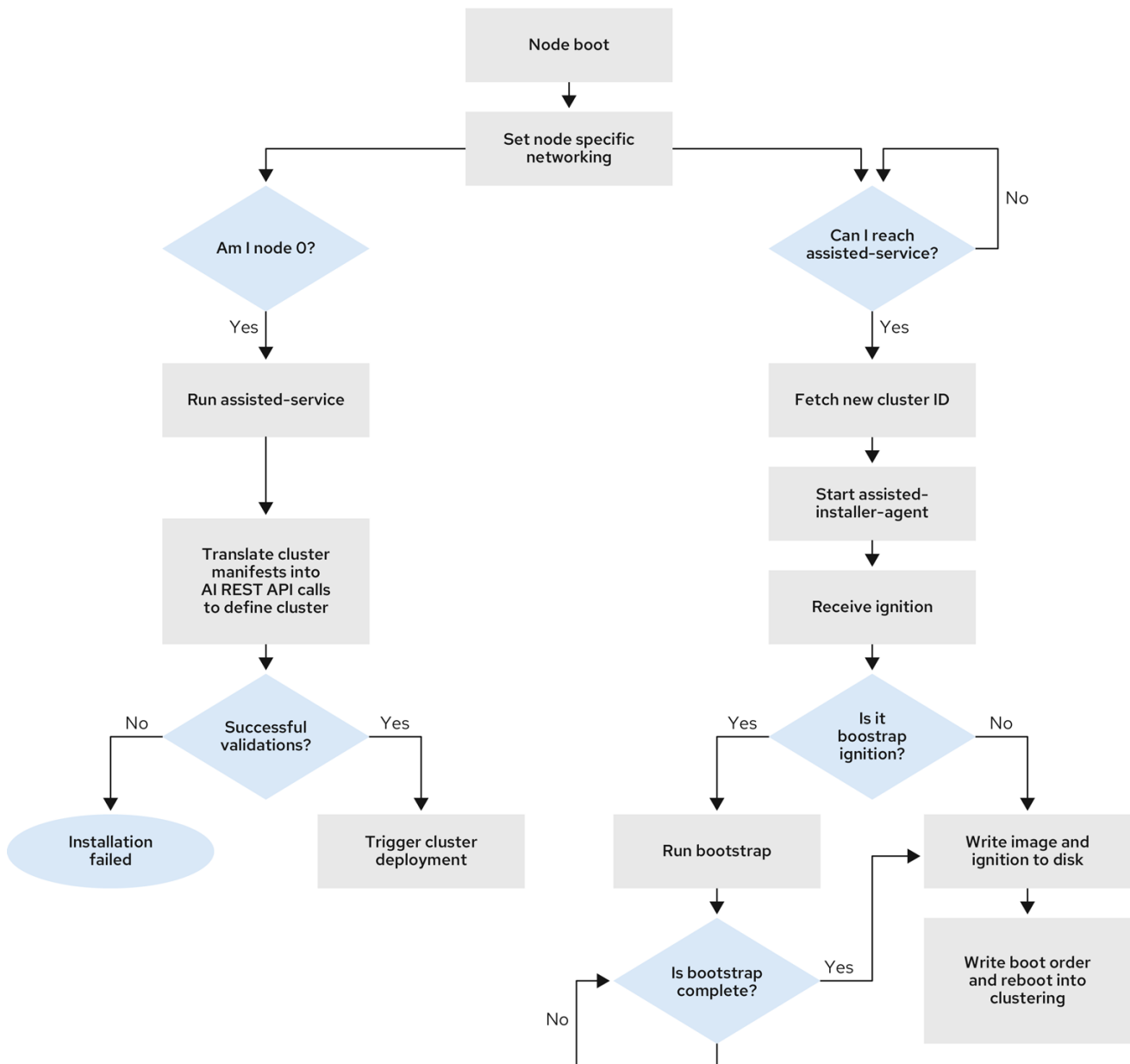
Optional: ZTP manifests

- **cluster-manifests/cluster-deployment.yaml**
- **cluster-manifests/agent-cluster-install.yaml**
- **cluster-manifests/pull-secret.yaml**
- **cluster-manifests/infraenv.yaml**
- **cluster-manifests/cluster-image-set.yaml**
- **cluster-manifests/nmstateconfig.yaml**
- **mirror/registries.conf**
- **mirror/ca-bundle.crt**

1.2.1. Agent-based Installer workflow

One of the control plane hosts runs the Assisted Service at the start of the boot process and eventually becomes the bootstrap host. This node is called the **rendezvous host** (node 0). The Assisted Service ensures that all the hosts meet the requirements and triggers an OpenShift Container Platform cluster deployment. All the nodes have the Red Hat Enterprise Linux CoreOS (RHCOS) image written to the disk. The non-bootstrap nodes reboot and initiate a cluster deployment. Once the nodes are rebooted, the rendezvous host reboots and joins the cluster. The bootstrapping is complete and the cluster is deployed.

Figure 1.1. Node installation workflow



281_OpenShift_1022

You can install a disconnected OpenShift Container Platform cluster through the **openshift-install agent create image** subcommand for the following topologies:

- **A single-node OpenShift Container Platform cluster (SNO)** A node that is both a master and worker.

- **A three-node OpenShift Container Platform cluster:** A compact cluster that has three master nodes that are also worker nodes.
- **Highly available OpenShift Container Platform cluster (HA)** Three master nodes with any number of worker nodes.

1.2.2. Recommended resources for topologies

Recommended cluster resources for the following topologies:

Table 1.1. Recommended cluster resources

Topology	Number of master nodes	Number of worker nodes	vCPU	Memory	Storage
Single-node cluster	1	0	8 vCPUs	16GB of RAM	120GB
Compact cluster	3	0 or 1	8 vCPUs	16GB of RAM	120GB
HA cluster	3	2 and above	8 vCPUs	16GB of RAM	120GB

The following platforms are supported:

- **baremetal**
- **vsphere**
- **external**
- **none**



IMPORTANT

For platform **none**:

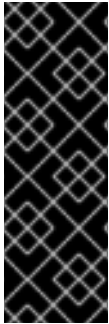
- The **none** option requires the provision of DNS name resolution and load balancing infrastructure in your cluster. See *Requirements for a cluster using the platform "none" option* in the "Additional resources" section for more information.
- Review the information in the [guidelines for deploying OpenShift Container Platform on non-tested platforms](#) before you attempt to install an OpenShift Container Platform cluster in virtualized or cloud environments.

Additional resources

- [Requirements for a cluster using the platform "none" option](#)
- [Increase the network MTU](#)
- [Adding worker nodes to single-node OpenShift clusters](#)

1.3. ABOUT FIPS COMPLIANCE

For many OpenShift Container Platform customers, regulatory readiness, or compliance, on some level is required before any systems can be put into production. That regulatory readiness can be imposed by national standards, industry standards or the organization's corporate governance framework. Federal Information Processing Standards (FIPS) compliance is one of the most critical components required in highly secure environments to ensure that only supported cryptographic technologies are allowed on nodes.



IMPORTANT

To enable FIPS mode for your cluster, you must run the installation program from a Red Hat Enterprise Linux (RHEL) computer configured to operate in FIPS mode. For more information about configuring FIPS mode on RHEL, see [Installing the system in FIPS mode](#). When running Red Hat Enterprise Linux (RHEL) or Red Hat Enterprise Linux CoreOS (RHCOS) booted in FIPS mode, OpenShift Container Platform core components use the RHEL cryptographic libraries that have been submitted to NIST for FIPS 140-2/140-3 Validation on only the x86_64, ppc64le, and s390x architectures.

1.4. CONFIGURING FIPS THROUGH THE AGENT-BASED INSTALLER

During a cluster deployment, the Federal Information Processing Standards (FIPS) change is applied when the Red Hat Enterprise Linux CoreOS (RHCOS) machines are deployed in your cluster. For Red Hat Enterprise Linux (RHEL) machines, you must enable FIPS mode when you install the operating system on the machines that you plan to use as worker machines.

You can enable FIPS mode through the preferred method of **install-config.yaml** and **agent-config.yaml**:

1. You must set value of the **fips** field to **true** in the **install-config.yaml** file:

Sample install-config.yaml file

```
apiVersion: v1
baseDomain: test.example.com
metadata:
  name: sno-cluster
fips: true
```

2. Optional: If you are using the GitOps ZTP manifests, you must set the value of **fips** as **true** in the **agent-install.openshift.io/install-config-overrides** field in the **agent-cluster-install.yaml** file:

Sample agent-cluster-install.yaml file

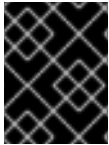
```
apiVersion: extensions.hive.openshift.io/v1beta1
kind: AgentClusterInstall
metadata:
  annotations:
    agent-install.openshift.io/install-config-overrides: '{"fips": true}'
  name: sno-cluster
  namespace: sno-cluster-test
```

Additional resources

- [OpenShift Security Guide Book](#)
- [Support for FIPS cryptography](#)

1.5. HOST CONFIGURATION

You can make additional configurations for each host on the cluster in the **agent-config.yaml** file, such as network configurations and root device hints.



IMPORTANT

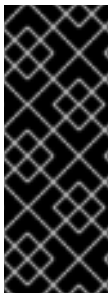
For each host you configure, you must provide the MAC address of an interface on the host to specify which host you are configuring.

1.5.1. Host roles

Each host in the cluster is assigned a role of either **master** or **worker**. You can define the role for each host in the **agent-config.yaml** file by using the **role** parameter. If you do not assign a role to the hosts, the roles will be assigned at random during installation.

It is recommended to explicitly define roles for your hosts.

The **rendezvousIP** must be assigned to a host with the **master** role. This can be done manually or by allowing the Agent-based Installer to assign the role.



IMPORTANT

You do not need to explicitly define the **master** role for the rendezvous host, however you cannot create configurations that conflict with this assignment.

For example, if you have 4 hosts with 3 of the hosts explicitly defined to have the **master** role, the last host that is automatically assigned the **worker** role during installation cannot be configured as the rendezvous host.

Sample agent-config.yaml file

```
apiVersion: v1beta1
kind: AgentConfig
metadata:
  name: example-cluster
rendezvousIP: 192.168.111.80
hosts:
  - hostname: master-1
    role: master
    interfaces:
      - name: eno1
        macAddress: 00:ef:44:21:e6:a5
  - hostname: master-2
    role: master
    interfaces:
      - name: eno1
        macAddress: 00:ef:44:21:e6:a6
  - hostname: master-3
    role: master
```

```

interfaces:
- name: eno1
  macAddress: 00:ef:44:21:e6:a7
- hostname: worker-1
  role: worker
  interfaces:
  - name: eno1
    macAddress: 00:ef:44:21:e6:a8

```

1.5.2. About root device hints

The **rootDeviceHints** parameter enables the installer to provision the Red Hat Enterprise Linux CoreOS (RHCOS) image to a particular device. The installer examines the devices in the order it discovers them, and compares the discovered values with the hint values. The installer uses the first discovered device that matches the hint value. The configuration can combine multiple hints, but a device must match all hints for the installer to select it.

Table 1.2. Subfields

Subfield	Description
deviceName	A string containing a Linux device name such as /dev/vda or /dev/disk/by-path/.  NOTE It is recommended to use the /dev/disk/by-path/<device_path> link to the storage location. The hint must match the actual value exactly.
hctl	A string containing a SCSI bus address like 0:0:0:0 . The hint must match the actual value exactly.
model	A string containing a vendor-specific device identifier. The hint can be a substring of the actual value.
vendor	A string containing the name of the vendor or manufacturer of the device. The hint can be a sub-string of the actual value.
serialNumber	A string containing the device serial number. The hint must match the actual value exactly.
minSizeGigabytes	An integer representing the minimum size of the device in gigabytes.
wwn	A string containing the unique storage identifier. The hint must match the actual value exactly. If you use the udevadm command to retrieve the wwn value, and the command outputs a value for ID_WWN_WITH_EXTENSION , then you must use this value to specify the wwn subfield.

Subfield	Description
rotational	A boolean indicating whether the device should be a rotating disk (true) or not (false).

Example usage

```
- name: master-0
  role: master
  rootDeviceHints:
    deviceName: "/dev/sda"
```

1.6. ABOUT NETWORKING

The **rendezvous IP** must be known at the time of generating the agent ISO, so that during the initial boot all the hosts can check in to the assisted service. If the IP addresses are assigned using a Dynamic Host Configuration Protocol (DHCP) server, then the **rendezvousIP** field must be set to an IP address of one of the hosts that will become part of the deployed control plane. In an environment without a DHCP server, you can define IP addresses statically.

In addition to static IP addresses, you can apply any network configuration that is in NMState format. This includes VLANs and NIC bonds.



NOTE

By default, Podman uses a subnet of **10.88.0.0/16** as a bridge network. Do not set the **network.machineNetwork.cidr** parameter to include this address range, otherwise a conflict causes the cluster installation to fail.

1.6.1. DHCP

Preferred method: `install-config.yaml` and `agent-config.yaml`

You must specify the value for the **rendezvousIP** field. The **networkConfig** fields can be left blank:

Sample `agent-config.yaml` file

```
apiVersion: v1alpha1
kind: AgentConfig
metadata:
  name: sno-cluster
rendezvousIP: 192.168.111.80 1
```

1

The IP address for the rendezvous host.

1.6.2. Static networking

a. Preferred method: **`install-config.yaml`** and **`agent-config.yaml`**

Sample `agent-config.yaml` file

```

cat > agent-config.yaml << EOF
apiVersion: v1alpha1
kind: AgentConfig
metadata:
  name: sno-cluster
rendezvousIP: 192.168.111.80 ❶
hosts:
- hostname: master-0
  interfaces:
  - name: eno1
    macAddress: 00:ef:44:21:e6:a5 ❷
  networkConfig:
    interfaces:
    - name: eno1
      type: ethernet
      state: up
      mac-address: 00:ef:44:21:e6:a5
      ipv4:
        enabled: true
        address:
        - ip: 192.168.111.80 ❸
          prefix-length: 23 ❹
        dhcp: false
      dns-resolver:
        config:
          server:
          - 192.168.111.1 ❺
        routes:
          config:
          - destination: 0.0.0.0/0
            next-hop-address: 192.168.111.1 ❻
            next-hop-interface: eno1
            table-id: 254
EOF

```

- ❶ If a value is not specified for the **rendezvousIP** field, one address will be chosen from the static IP addresses specified in the **networkConfig** fields.
- ❷ The MAC address of an interface on the host, used to determine which host to apply the configuration to.
- ❸ The static IP address of the target bare metal host.
- ❹ The static IP address's subnet prefix for the target bare metal host.
- ❺ The DNS server for the target bare metal host.
- ❻ Next hop address for the node traffic. This must be in the same subnet as the IP address set for the specified interface.

b. Optional method: GitOps ZTP manifests

The optional method of the GitOps ZTP custom resources comprises 6 custom resources; you can configure static IPs in the **nmstateconfig.yaml** file.

```

apiVersion: agent-install.openshift.io/v1beta1
kind: NMStateConfig
metadata:
  name: master-0
  namespace: openshift-machine-api
  labels:
    cluster0-nmstate-label-name: cluster0-nmstate-label-value
spec:
  config:
    interfaces:
      - name: eth0
        type: ethernet
        state: up
        mac-address: 52:54:01:aa:aa:a1
        ipv4:
          enabled: true
          address:
            - ip: 192.168.122.2 ❶
              prefix-length: 23 ❷
          dhcp: false
        dns-resolver:
          config:
            server:
              - 192.168.122.1 ❸
        routes:
          config:
            - destination: 0.0.0.0/0
              next-hop-address: 192.168.122.1 ❹
              next-hop-interface: eth0
              table-id: 254
    interfaces:
      - name: eth0
        macAddress: 52:54:01:aa:aa:a1 ❺

```

- ❶ The static IP address of the target bare metal host.
- ❷ The static IP address's subnet prefix for the target bare metal host.
- ❸ The DNS server for the target bare metal host.
- ❹ Next hop address for the node traffic. This must be in the same subnet as the IP address set for the specified interface.
- ❺ The MAC address of an interface on the host, used to determine which host to apply the configuration to.

The rendezvous IP is chosen from the static IP addresses specified in the **config** fields.

1.7. REQUIREMENTS FOR A CLUSTER USING THE PLATFORM "NONE" OPTION

This section describes the requirements for an Agent-based OpenShift Container Platform installation that is configured to use the platform **none** option.



IMPORTANT

Review the information in the [guidelines for deploying OpenShift Container Platform on non-tested platforms](#) before you attempt to install an OpenShift Container Platform cluster in virtualized or cloud environments.

1.7.1. Platform "none" DNS requirements

In OpenShift Container Platform deployments, DNS name resolution is required for the following components:

- The Kubernetes API
- The OpenShift Container Platform application wildcard
- The control plane and compute machines

Reverse DNS resolution is also required for the Kubernetes API, the control plane machines, and the compute machines.

DNS A/AAAA or CNAME records are used for name resolution and PTR records are used for reverse name resolution. The reverse records are important because Red Hat Enterprise Linux CoreOS (RHCOS) uses the reverse records to set the hostnames for all the nodes, unless the hostnames are provided by DHCP. Additionally, the reverse records are used to generate the certificate signing requests (CSR) that OpenShift Container Platform needs to operate.



NOTE

It is recommended to use a DHCP server to provide the hostnames to each cluster node.

The following DNS records are required for an OpenShift Container Platform cluster using the platform **none** option and they must be in place before installation. In each record, **<cluster_name>** is the cluster name and **<base_domain>** is the base domain that you specify in the **install-config.yaml** file. A complete DNS record takes the form: **<component>.<cluster_name>.<base_domain>..**

Table 1.3. Required DNS records

Component	Record	Description
Kubernetes API	api.<cluster_name>.<base_domain>.	A DNS A/AAAA or CNAME record, and a DNS PTR record, to identify the API load balancer. These records must be resolvable by both clients external to the cluster and from all the nodes within the cluster.

Component	Record	Description
	api-int.<cluster_name>.<base_domain>.	A DNS A/AAAA or CNAME record, and a DNS PTR record, to internally identify the API load balancer. These records must be resolvable from all the nodes within the cluster.  IMPORTANT The API server must be able to resolve the worker nodes by the hostnames that are recorded in Kubernetes. If the API server cannot resolve the node names, then proxied API calls can fail, and you cannot retrieve logs from pods.
Routes	*.apps.<cluster_name>.<base_domain>.	A wildcard DNS A/AAAA or CNAME record that refers to the application ingress load balancer. The application ingress load balancer targets the machines that run the Ingress Controller pods. The Ingress Controller pods run on the compute machines by default. These records must be resolvable by both clients external to the cluster and from all the nodes within the cluster. For example, console-openshift-console.apps.<cluster_name>.<base_domain> is used as a wildcard route to the OpenShift Container Platform console.
Control plane machines	<master><n>.<cluster_name>.<base_domain>.	DNS A/AAAA or CNAME records and DNS PTR records to identify each machine for the control plane nodes. These records must be resolvable by the nodes within the cluster.
Compute machines	<worker><n>.<cluster_name>.<base_domain>.	DNS A/AAAA or CNAME records and DNS PTR records to identify each machine for the worker nodes. These records must be resolvable by the nodes within the cluster.

**NOTE**

In OpenShift Container Platform 4.4 and later, you do not need to specify etcd host and SRV records in your DNS configuration.

TIP

You can use the **dig** command to verify name and reverse name resolution.

1.7.1.1. Example DNS configuration for platform "none" clusters

This section provides A and PTR record configuration samples that meet the DNS requirements for deploying OpenShift Container Platform using the platform **none** option. The samples are not meant to provide advice for choosing one DNS solution over another.

In the examples, the cluster name is **ocp4** and the base domain is **example.com**.

Example DNS A record configuration for a platform "none" cluster

The following example is a BIND zone file that shows sample A records for name resolution in a cluster using the platform **none** option.

Example 1.1. Sample DNS zone database

```
$TTL 1W
@ IN SOA ns1.example.com. root (
    2019070700 ; serial
    3H ; refresh (3 hours)
    30M ; retry (30 minutes)
    2W ; expiry (2 weeks)
    1W ) ; minimum (1 week)
IN NS ns1.example.com.
IN MX 10 smtp.example.com.
;
;
ns1.example.com. IN A 192.168.1.5
smtp.example.com. IN A 192.168.1.5
;
helper.example.com. IN A 192.168.1.5
helper.ocp4.example.com. IN A 192.168.1.5
;
api.ocp4.example.com. IN A 192.168.1.5 1
api-int.ocp4.example.com. IN A 192.168.1.5 2
;
*.apps.ocp4.example.com. IN A 192.168.1.5 3
;
master0.ocp4.example.com. IN A 192.168.1.97 4
master1.ocp4.example.com. IN A 192.168.1.98 5
master2.ocp4.example.com. IN A 192.168.1.99 6
;
worker0.ocp4.example.com. IN A 192.168.1.11 7
worker1.ocp4.example.com. IN A 192.168.1.7 8
;
;EOF
```

- 1 Provides name resolution for the Kubernetes API. The record refers to the IP address of the API load balancer.
- 2 Provides name resolution for the Kubernetes API. The record refers to the IP address of the API load balancer and is used for internal cluster communications.
- 3 Provides name resolution for the wildcard routes. The record refers to the IP address of the application ingress load balancer. The application ingress load balancer targets the machines that run the Ingress Controller pods. The Ingress Controller pods run on the compute machines

by default.



NOTE

In the example, the same load balancer is used for the Kubernetes API and application ingress traffic. In production scenarios, you can deploy the API and application ingress load balancers separately so that you can scale the load balancer infrastructure for each in isolation.

4 5 6 Provides name resolution for the control plane machines.

7 8 Provides name resolution for the compute machines.

Example DNS PTR record configuration for a platform "none" cluster

The following example BIND zone file shows sample PTR records for reverse name resolution in a cluster using the platform **none** option.

Example 1.2. Sample DNS zone database for reverse records

```
$TTL 1W
@ IN SOA ns1.example.com. root (
    2019070700 ; serial
    3H ; refresh (3 hours)
    30M ; retry (30 minutes)
    2W ; expiry (2 weeks)
    1W ) ; minimum (1 week)
IN NS ns1.example.com.
;
5.1.168.192.in-addr.arpa. IN PTR api.ocp4.example.com. 1
5.1.168.192.in-addr.arpa. IN PTR api-int.ocp4.example.com. 2
;
97.1.168.192.in-addr.arpa. IN PTR master0.ocp4.example.com. 3
98.1.168.192.in-addr.arpa. IN PTR master1.ocp4.example.com. 4
99.1.168.192.in-addr.arpa. IN PTR master2.ocp4.example.com. 5
;
11.1.168.192.in-addr.arpa. IN PTR worker0.ocp4.example.com. 6
7.1.168.192.in-addr.arpa. IN PTR worker1.ocp4.example.com. 7
;
;EOF
```

1 Provides reverse DNS resolution for the Kubernetes API. The PTR record refers to the record name of the API load balancer.

2 Provides reverse DNS resolution for the Kubernetes API. The PTR record refers to the record name of the API load balancer and is used for internal cluster communications.

3 4 5 Provides reverse DNS resolution for the control plane machines.

6 7 Provides reverse DNS resolution for the compute machines.

**NOTE**

A PTR record is not required for the OpenShift Container Platform application wildcard.

1.7.2. Platform "none" Load balancing requirements

Before you install OpenShift Container Platform, you must provision the API and application Ingress load balancing infrastructure. In production scenarios, you can deploy the API and application Ingress load balancers separately so that you can scale the load balancer infrastructure for each in isolation.

**NOTE**

These requirements do not apply to single-node OpenShift clusters using the platform **none** option.

**NOTE**

If you want to deploy the API and application Ingress load balancers with a Red Hat Enterprise Linux (RHEL) instance, you must purchase the RHEL subscription separately.

The load balancing infrastructure must meet the following requirements:

1. **API load balancer.** Provides a common endpoint for users, both human and machine, to interact with and configure the platform. Configure the following conditions:
 - Layer 4 load balancing only. This can be referred to as Raw TCP, SSL Passthrough, or SSL Bridge mode. If you use SSL Bridge mode, you must enable Server Name Indication (SNI) for the API routes.
 - A stateless load balancing algorithm. The options vary based on the load balancer implementation.

**IMPORTANT**

Do not configure session persistence for an API load balancer.

Configure the following ports on both the front and back of the load balancers:

Table 1.4. API load balancer

Port	Back-end machines (pool members)	Internal	External	Description
6443	Control plane. You must configure the /readyz endpoint for the API server health check probe.	X	X	Kubernetes API server
22623	Control plane.	X		Machine config server

**NOTE**

The load balancer must be configured to take a maximum of 30 seconds from the time the API server turns off the **/readyz** endpoint to the removal of the API server instance from the pool. Within the time frame after **/readyz** returns an error or becomes healthy, the endpoint must have been removed or added. Probing every 5 or 10 seconds, with two successful requests to become healthy and three to become unhealthy, are well-tested values.

2. **Application Ingress load balancer:** Provides an ingress point for application traffic flowing in from outside the cluster. A working configuration for the Ingress router is required for an OpenShift Container Platform cluster.

Configure the following conditions:

- Layer 4 load balancing only. This can be referred to as Raw TCP, SSL Passthrough, or SSL Bridge mode. If you use SSL Bridge mode, you must enable Server Name Indication (SNI) for the ingress routes.
- A connection-based or session-based persistence is recommended, based on the options available and types of applications that will be hosted on the platform.

TIP

If the true IP address of the client can be seen by the application Ingress load balancer, enabling source IP-based session persistence can improve performance for applications that use end-to-end TLS encryption.

Configure the following ports on both the front and back of the load balancers:

Table 1.5. Application Ingress load balancer

Port	Back-end machines (pool members)	Internal	External	Description
443	The machines that run the Ingress Controller pods, compute, or worker, by default.	X	X	HTTPS traffic
80	The machines that run the Ingress Controller pods, compute, or worker, by default.	X	X	HTTP traffic

**NOTE**

If you are deploying a three-node cluster with zero compute nodes, the Ingress Controller pods run on the control plane nodes. In three-node cluster deployments, you must configure your application Ingress load balancer to route HTTP and HTTPS traffic to the control plane nodes.

1.7.2.1. Example load balancer configuration for platform "none" clusters

This section provides an example API and application Ingress load balancer configuration that meets the load balancing requirements for clusters using the platform **none** option. The sample is an **/etc/haproxy/haproxy.cfg** configuration for an HAProxy load balancer. The example is not meant to

provide advice for choosing one load balancing solution over another.

In the example, the same load balancer is used for the Kubernetes API and application ingress traffic. In production scenarios, you can deploy the API and application ingress load balancers separately so that you can scale the load balancer infrastructure for each in isolation.



NOTE

If you are using HAProxy as a load balancer and SELinux is set to **enforcing**, you must ensure that the HAProxy service can bind to the configured TCP port by running **setsebool -P haproxy_connect_any=1**.

Example 1.3. Sample API and application Ingress load balancer configuration

```
global
  log      127.0.0.1 local2
  pidfile  /var/run/haproxy.pid
  maxconn  4000
  daemon
defaults
  mode                http
  log                 global
  option              dontlognull
  option http-server-close
  option              redispatch
  retries              3
  timeout http-request 10s
  timeout queue        1m
  timeout connect      10s
  timeout client        1m
  timeout server        1m
  timeout http-keep-alive 10s
  timeout check         10s
  maxconn              3000
listen api-server-6443 1
  bind *:6443
  mode tcp
  server master0 master0.ocp4.example.com:6443 check inter 1s
  server master1 master1.ocp4.example.com:6443 check inter 1s
  server master2 master2.ocp4.example.com:6443 check inter 1s
listen machine-config-server-22623 2
  bind *:22623
  mode tcp
  server master0 master0.ocp4.example.com:22623 check inter 1s
  server master1 master1.ocp4.example.com:22623 check inter 1s
  server master2 master2.ocp4.example.com:22623 check inter 1s
listen ingress-router-443 3
  bind *:443
  mode tcp
  balance source
  server worker0 worker0.ocp4.example.com:443 check inter 1s
  server worker1 worker1.ocp4.example.com:443 check inter 1s
listen ingress-router-80 4
  bind *:80
```

```
mode tcp
balance source
server worker0 worker0.ocp4.example.com:80 check inter 1s
server worker1 worker1.ocp4.example.com:80 check inter 1s
```

- 1 Port **6443** handles the Kubernetes API traffic and points to the control plane machines.
- 2 Port **22623** handles the machine config server traffic and points to the control plane machines.
- 3 Port **443** handles the HTTPS traffic and points to the machines that run the Ingress Controller pods. The Ingress Controller pods run on the compute machines by default.
- 4 Port **80** handles the HTTP traffic and points to the machines that run the Ingress Controller pods. The Ingress Controller pods run on the compute machines by default.



NOTE

If you are deploying a three-node cluster with zero compute nodes, the Ingress Controller pods run on the control plane nodes. In three-node cluster deployments, you must configure your application Ingress load balancer to route HTTP and HTTPS traffic to the control plane nodes.

TIP

If you are using HAProxy as a load balancer, you can check that the **haproxy** process is listening on ports **6443**, **22623**, **443**, and **80** by running **netstat -nltpu** on the HAProxy node.

1.8. EXAMPLE: BONDS AND VLAN INTERFACE NODE NETWORK CONFIGURATION

The following **agent-config.yaml** file is an example of a manifest for bond and VLAN interfaces.

```
apiVersion: v1alpha1
kind: AgentConfig
rendezvousIP: 10.10.10.14
hosts:
- hostname: master0
  role: master
  interfaces:
  - name: enp0s4
    macAddress: 00:21:50:90:c0:10
  - name: enp0s5
    macAddress: 00:21:50:90:c0:20
networkConfig:
  interfaces:
  - name: bond0.300 1
    type: vlan 2
    state: up
    vlan:
      base-iface: bond0
      id: 300
```



```

    ipv4:
      enabled: true
      address:
        - ip: 10.10.10.14
          prefix-length: 24
      dhcp: false
  - name: bond0 3
    type: bond 4
    state: up
    mac-address: 00:21:50:90:c0:10 5
    ipv4:
      enabled: false
    ipv6:
      enabled: false
    link-aggregation:
      mode: active-backup 6
      options:
        miimon: "150" 7
      port:
        - enp0s4
        - enp0s5
    dns-resolver: 8
    config:
      server:
        - 10.10.10.11
        - 10.10.10.12
    routes:
      config:
        - destination: 0.0.0.0/0
          next-hop-address: 10.10.10.10 9
          next-hop-interface: bond0.300 10
          table-id: 254

```

- 1 3 Name of the interface.
- 2 The type of interface. This example creates a VLAN.
- 4 The type of interface. This example creates a bond.
- 5 The mac address of the interface.
- 6 The **mode** attribute specifies the bonding mode.
- 7 Specifies the MII link monitoring frequency in milliseconds. This example inspects the bond link every 150 milliseconds.
- 8 Optional: Specifies the search and server settings for the DNS server.
- 9 Next hop address for the node traffic. This must be in the same subnet as the IP address set for the specified interface.
- 10 Next hop interface for the node traffic.

1.9. EXAMPLE: BONDS AND SR-IOV DUAL-NIC NODE NETWORK CONFIGURATION



IMPORTANT

Support for Day 1 operations associated with enabling NIC partitioning for SR-IOV devices is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

The following **agent-config.yaml** file is an example of a manifest for dual port network interface controller (NIC) with a bond and SR-IOV interfaces:

```
apiVersion: v1alpha1
kind: AgentConfig
rendezvousIP: 10.10.10.14
hosts:
- hostname: worker-1
  interfaces:
  - name: eno1
    macAddress: 0c:42:a1:55:f3:06
  - name: eno2
    macAddress: 0c:42:a1:55:f3:07
networkConfig: 1
  interfaces: 2
  - name: eno1 3
    type: ethernet 4
    state: up
    mac-address: 0c:42:a1:55:f3:06
    ipv4:
      enabled: true
      dhcp: false 5
    ethernet:
      sr-iov:
        total-vfs: 2 6
    ipv6:
      enabled: false
  - name: sriov:eno1:0
    type: ethernet
    state: up 7
    ipv4:
      enabled: false 8
    ipv6:
      enabled: false
      dhcp: false
  - name: sriov:eno1:1
    type: ethernet
    state: down
  - name: eno2
```

```

type: ethernet
state: up
mac-address: 0c:42:a1:55:f3:07
ipv4:
  enabled: true
ethernet:
  sr-iov:
    total-vfs: 2
ipv6:
  enabled: false
- name: sriov:eno2:0
type: ethernet
state: up
ipv4:
  enabled: false
ipv6:
  enabled: false
- name: sriov:eno2:1
type: ethernet
state: down
- name: bond0
type: bond
state: up
min-tx-rate: 100 9
max-tx-rate: 200 10
link-aggregation:
  mode: active-backup 11
  options:
    primary: sriov:eno1:0 12
  port:
    - sriov:eno1:0
    - sriov:eno2:0
ipv4:
  address:
    - ip: 10.19.16.57 13
    prefix-length: 23
  dhcp: false
  enabled: true
ipv6:
  enabled: false
dns-resolver:
  config:
    server:
      - 10.11.5.160
      - 10.2.70.215
routes:
  config:
    - destination: 0.0.0.0/0
      next-hop-address: 10.19.17.254
      next-hop-interface: bond0 14
      table-id: 254

```

1

The **networkConfig** field includes information about the network configuration of the host, with subfields including **interfaces**, **dns-resolver**, and **routes**.

- 2 The **interfaces** field is an array of network interfaces defined for the host.
- 3 The name of the interface.
- 4 The type of interface. This example creates an ethernet interface.
- 5 Set this to **false** to disable DHCP for the physical function (PF) if it is not strictly required.
- 6 Set this to the number of SR-IOV virtual functions (VFs) to instantiate.
- 7 Set this to **up**.
- 8 Set this to **false** to disable IPv4 addressing for the VF attached to the bond.
- 9 Sets a minimum transmission rate, in Mbps, for the VF. This sample value sets a rate of 100 Mbps.
 - This value must be less than or equal to the maximum transmission rate.
 - Intel NICs do not support the **min-tx-rate** parameter. For more information, see [BZ#1772847](#).
- 10 Sets a maximum transmission rate, in Mbps, for the VF. This sample value sets a rate of 200 Mbps.
- 11 Sets the needed bond mode.
- 12 Sets the preferred port of the bonding interface. The primary device is the first of the bonding interfaces to be used and is not abandoned unless it fails. This setting is particularly useful when one NIC in the bonding interface is faster and, therefore, able to handle a bigger load. This setting is only valid when the bonding interface is in **active-backup** mode (mode 1).
- 13 Sets a static IP address for the bond interface. This is the node IP address.
- 14 Sets **bond0** as the gateway for the default route.

Additional resources

- [Configuring network bonding](#)

1.10. SAMPLE INSTALL-CONFIG.YAML FILE FOR BARE METAL

You can customize the **install-config.yaml** file to specify more details about your OpenShift Container Platform cluster's platform or modify the values of the required parameters.

```
apiVersion: v1
baseDomain: example.com 1
compute: 2
- name: worker
  replicas: 0 3
controlPlane: 4
  name: master
  replicas: 1 5
metadata:
  name: sno-cluster 6
networking:
```

```

clusterNetwork:
- cidr: 10.128.0.0/14 7
  hostPrefix: 23 8
machineNetwork:
- cidr: 192.168.0.0/16
networkType: OVNKubernetes 9
serviceNetwork: 10
- 172.30.0.0/16
platform:
  none: {} 11
fips: false 12
pullSecret: '{"auths": ...}' 13
sshKey: 'ssh-ed25519 AAAA...' 14

```

- 1 The base domain of the cluster. All DNS records must be sub-domains of this base and include the cluster name.
- 2 4 The **controlPlane** section is a single mapping, but the **compute** section is a sequence of mappings. To meet the requirements of the different data structures, the first line of the **compute** section must begin with a hyphen, -, and the first line of the **controlPlane** section must not. Only one control plane pool is used.
- 3 This parameter controls the number of compute machines that the Agent-based installation waits to discover before triggering the installation process. It is the number of compute machines that must be booted with the generated ISO.



NOTE

If you are installing a three-node cluster, do not deploy any compute machines when you install the Red Hat Enterprise Linux CoreOS (RHCOS) machines.

- 5 The number of control plane machines that you add to the cluster. Because the cluster uses these values as the number of etcd endpoints in the cluster, the value must match the number of control plane machines that you deploy.
- 6 The cluster name that you specified in your DNS records.
- 7 A block of IP addresses from which pod IP addresses are allocated. This block must not overlap with existing physical networks. These IP addresses are used for the pod network. If you need to access the pods from an external network, you must configure load balancers and routers to manage the traffic.



NOTE

Class E CIDR range is reserved for a future use. To use the Class E CIDR range, you must ensure your networking environment accepts the IP addresses within the Class E CIDR range.

- 8 The subnet prefix length to assign to each individual node. For example, if **hostPrefix** is set to **23**, then each node is assigned a /23 subnet out of the given **cidr**, which allows for 510 ($2^{(32 - 23)} - 2$) pod IP addresses. If you are required to provide access to nodes from an external network, configure load balancers and routers to manage the traffic.

- 9 The cluster network plugin to install. The supported values are **OVNKubernetes** (default value) and **OpenShiftSDN**.
- 10 The IP address pool to use for service IP addresses. You can enter only one IP address pool. This block must not overlap with existing physical networks. If you need to access the services from an external network, configure load balancers and routers to manage the traffic.
- 11 You must set the platform to **none** for a single-node cluster. You can set the platform to **vsphere**, **baremetal**, or **none** for multi-node clusters.

NOTE

If you set the platform to **vsphere** or **baremetal**, you can configure IP address endpoints for cluster nodes in three ways:

- IPv4
- IPv6
- IPv4 and IPv6 in parallel (dual-stack)

Example of dual-stack networking

```
networking:
  clusterNetwork:
    - cidr: 172.21.0.0/16
      hostPrefix: 23
    - cidr: fd02::/48
      hostPrefix: 64
  machineNetwork:
    - cidr: 192.168.11.0/16
    - cidr: 2001:DB8::/32
  serviceNetwork:
    - 172.22.0.0/16
    - fd03::/112
  networkType: OVNKubernetes
platform:
  baremetal:
    apiVIPs:
      - 192.168.11.3
      - 2001:DB8::4
    ingressVIPs:
      - 192.168.11.4
      - 2001:DB8::5
```

- 12 Whether to enable or disable FIPS mode. By default, FIPS mode is not enabled. If FIPS mode is enabled, the Red Hat Enterprise Linux CoreOS (RHCOS) machines that OpenShift Container Platform runs on bypass the default Kubernetes cryptography suite and use the cryptography modules that are provided with RHCOS instead.



IMPORTANT

When running Red Hat Enterprise Linux (RHEL) or Red Hat Enterprise Linux CoreOS (RHCOS) booted in FIPS mode, OpenShift Container Platform core components use the RHEL cryptographic libraries that have been submitted to NIST for FIPS 140-2/140-3 Validation on only the x86_64, ppc64le, and s390x architectures.

- 13 This pull secret allows you to authenticate with the services that are provided by the included authorities, including Quay.io, which serves the container images for OpenShift Container Platform components.
- 14 The SSH public key for the **core** user in Red Hat Enterprise Linux CoreOS (RHCOS).



NOTE

For production OpenShift Container Platform clusters on which you want to perform installation debugging or disaster recovery, specify an SSH key that your **ssh-agent** process uses.

1.11. VALIDATION CHECKS BEFORE AGENT ISO CREATION

The Agent-based Installer performs validation checks on user defined YAML files before the ISO is created. Once the validations are successful, the agent ISO is created.

install-config.yaml

- **baremetal**, **vsphere** and **none** platforms are supported.
- The **networkType** parameter must be **OVNKubernetes** in the case of **none** platform.
- **apiVIPs** and **ingressVIPs** parameters must be set for bare metal and vSphere platforms.
- Some host-specific fields in the bare metal platform configuration that have equivalents in **agent-config.yaml** file are ignored. A warning message is logged if these fields are set.

agent-config.yaml

- Each interface must have a defined MAC address. Additionally, all interfaces must have a different MAC address.
- At least one interface must be defined for each host.
- World Wide Name (WWN) vendor extensions are not supported in root device hints.
- The **role** parameter in the **host** object must have a value of either **master** or **worker**.

1.11.1. ZTP manifests

agent-cluster-install.yaml

- For IPv6, the only supported value for the **networkType** parameter is **OVNKubernetes**. The **OpenshiftSDN** value can be used only for IPv4.

cluster-image-set.yaml

- The **ReleaseImage** parameter must match the release defined in the installer.

1.12. NEXT STEPS

- [Installing a cluster](#)
- [Installing a cluster with customizations](#)

CHAPTER 2. UNDERSTANDING DISCONNECTED INSTALLATION MIRRORING

You can use a mirror registry for disconnected installations and to ensure that your clusters only use container images that satisfy your organization's controls on external content. Before you install a cluster on infrastructure that you provision in a disconnected environment, you must mirror the required container images into that environment. To mirror container images, you must have a registry for mirroring.

2.1. MIRRORING IMAGES FOR A DISCONNECTED INSTALLATION THROUGH THE AGENT-BASED INSTALLER

You can use one of the following procedures to mirror your OpenShift Container Platform image repository to your mirror registry:

- [Mirroring images for a disconnected installation](#)
- [Mirroring images for a disconnected installation using the oc-mirror plugin](#)

2.2. ABOUT MIRRORING THE OPENSIFT CONTAINER PLATFORM IMAGE REPOSITORY FOR A DISCONNECTED REGISTRY

To use mirror images for a disconnected installation with the Agent-based Installer, you must modify the **install-config.yaml** file.

You can mirror the release image by using the output of either the **oc adm release mirror** or **oc mirror** command. This is dependent on which command you used to set up the mirror registry.

The following example shows the output of the **oc adm release mirror** command.

```
$ oc adm release mirror
```

Example output

To use the new mirrored repository to install, add the following section to the **install-config.yaml**:

```
imageContentSources:
```

```
  mirrors:
```

```
    virthost.ostest.test.metalkube.org:5000/localimages/local-release-image
```

```
    source: quay.io/openshift-release-dev/ocp-v4.0-art-dev
```

```
  mirrors:
```

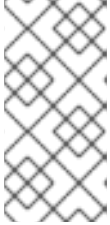
```
    virthost.ostest.test.metalkube.org:5000/localimages/local-release-image
```

```
    source: registry.ci.openshift.org/ocp/release
```

The following example shows part of the **imageContentSourcePolicy.yaml** file generated by the **oc-mirror** plugin. The file can be found in the results directory, for example **oc-mirror-workspace/results-1682697932/**.

Example imageContentSourcePolicy.yaml file

```
■
```

NOTE

You can create the **registries.conf** file from the output of either the **oc adm release mirror** command or the **oc mirror** plugin. The format of the **/etc/containers/registries.conf** file has changed. It is now version 2 and in TOML format.

Example registries.conf file

```
[[registry]]
location = "registry.ci.openshift.org/ocp/release" mirror-by-digest-only = true

[[registry.mirror]] location = "virthost.ostest.test.metalkube.org:5000/localimages/local-release-image"

[[registry]]
location = "quay.io/openshift-release-dev/ocp-v4.0-art-dev" mirror-by-digest-only = true

[[registry.mirror]] location = "virthost.ostest.test.metalkube.org:5000/localimages/local-release-image"
```

CHAPTER 3. INSTALLING A CLUSTER

You can install a basic OpenShift Container Platform cluster using the Agent-based Installer.

For procedures that include optional customizations you can make while using the Agent-based Installer, see [Installing a cluster with customizations](#).

3.1. PREREQUISITES

- You reviewed details about the [OpenShift Container Platform installation and update](#) processes.
- You read the documentation on [selecting a cluster installation method and preparing it for users](#).
- If you use a firewall or proxy, you [configured it to allow the sites](#) that your cluster requires access to.

3.2. INSTALLING OPENSIFT CONTAINER PLATFORM WITH THE AGENT-BASED INSTALLER

The following procedures deploy a single-node OpenShift Container Platform in a disconnected environment. You can use these procedures as a basis and modify according to your requirements.

3.2.1. Downloading the Agent-based Installer

Use this procedure to download the Agent-based Installer and the CLI needed for your installation.

Procedure

1. Log in to the Red Hat Hybrid Cloud Console using your login credentials.
2. Navigate to [Datacenter](#).
3. Click **Run Agent-based Installer locally**.
4. Select the operating system and architecture for the **OpenShift Installer** and **Command line interface**.
5. Click **Download Installer** to download and extract the install program.
6. Download or copy the pull secret by clicking on **Download pull secret** or **Copy pull secret**
7. Click **Download command-line tools** and place the **openshift-install** binary in a directory that is on your **PATH**.

3.2.2. Creating the configuration inputs

You must create the configuration files that are used by the installation program to create the agent image.

Procedure

1. Place the **openshift-install** binary in a directory that is on your PATH.

2. Create a directory to store the install configuration by running the following command:

```
$ mkdir ~/<directory_name>
```

3. Create the **install-config.yaml** file by running the following command:

[illegible]

- 1 Specify the system architecture. Valid values are **amd64**, **arm64**, **ppc64le**, and **s390x**.

If you are using the release image with the **multi** payload, you can install the cluster on different architectures such as **arm64**, **amd64**, **s390x**, and **ppc64le**. Otherwise, you can install the cluster only on the **release architecture** displayed in the output of the **openshift-install version** command. For more information, see "Verifying the supported architecture for installing an Agent-based Installer cluster".

- 2 Required. Specify your cluster name.
- 3 The cluster network plugin to install. The default value **OVNKubernetes** is the only supported value.
- 4 Specify your platform.



NOTE

For bare metal platforms, host settings made in the platform section of the **install-config.yaml** file are used by default, unless they are overridden by configurations made in the **agent-config.yaml** file.

- 5 Specify your pull secret.
- 6 Specify your SSH public key.
- 7 Provide the contents of the certificate file that you used for your mirror registry. The certificate file can be an existing, trusted certificate authority or the self-signed certificate that you generated for the mirror registry. You must specify this parameter if you are using a disconnected mirror registry.
- 8 Provide the **imageContentSources** section according to the output of the command that you used to mirror the repository. You must specify this parameter if you are using a disconnected mirror registry.



IMPORTANT

- When using the **oc adm release mirror** command, use the output from the **imageContentSources** section.
- When using the **oc mirror** command, use the **repositoryDigestMirrors** section of the **ImageContentSourcePolicy** file that results from running the command.
- The **ImageContentSourcePolicy** resource is deprecated.

4. Create the **agent-config.yaml** file by running the following command:

```
$ cat > agent-config.yaml << EOF
apiVersion: v1beta1
kind: AgentConfig
metadata:
  name: sno-cluster
rendezvousIP: fd2e:6f44:5dd8:c956::50 1
EOF
```

- 1 This IP address is used to determine which node performs the bootstrapping process as well as running the **assisted-service** component. You must provide the rendezvous IP address when you do not specify at least one host IP address in the **networkConfig** parameter. If this address is not provided, one IP address is selected from the provided host **networkConfig** parameter.

3.2.3. Creating and booting the agent image

Use this procedure to boot the agent image on your machines.

Procedure

1. Create the agent image by running the following command:

```
$ openshift-install --dir <install_directory> agent create image
```



NOTE

Red Hat Enterprise Linux CoreOS (RHCOS) supports multipathing on the primary disk, allowing stronger resilience to hardware failure to achieve higher host availability. Multipathing is enabled by default in the agent ISO image, with a default **/etc/multipath.conf** configuration.

2. Boot the **agent.x86_64.iso** or **agent.aarch64.iso** image on the bare metal machines.

3.2.4. Verifying that the current installation host can pull release images

After you boot the agent image and network services are made available to the host, the agent console application performs a pull check to verify that the current host can retrieve release images.

If the primary pull check passes, you can quit the application to continue with the installation. If the pull check fails, the application performs additional checks, as seen in the **Additional checks** section of the TUI, to help you troubleshoot the problem. A failure for any of the additional checks is not necessarily critical as long as the primary pull check succeeds.

If there are host network configuration issues that might cause an installation to fail, you can use the console application to make adjustments to your network configurations.



IMPORTANT

If the agent console application detects host network configuration issues, the installation workflow will be halted until the user manually stops the console application and signals the intention to proceed.

Procedure

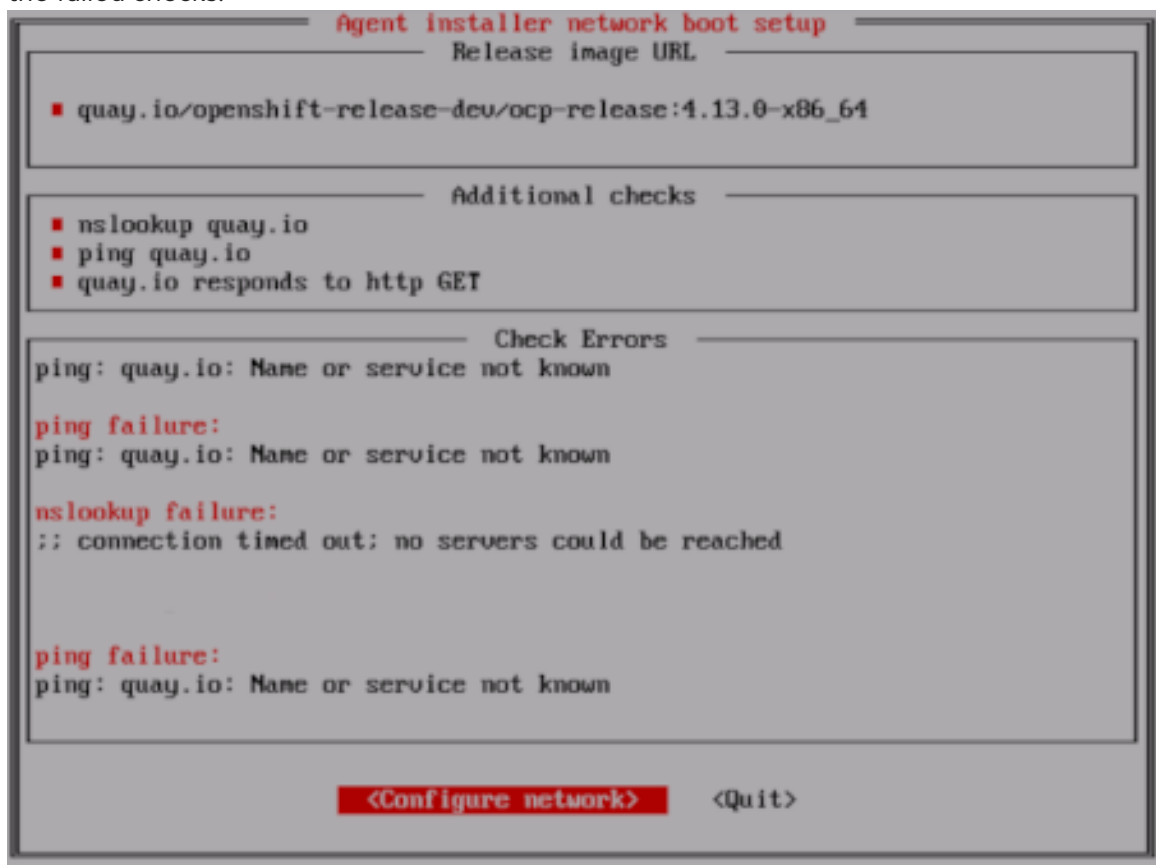
1. Wait for the agent console application to check whether or not the configured release image can be pulled from a registry.
2. If the agent console application states that the installer connectivity checks have passed, wait for the prompt to time out to continue with the installation.

**NOTE**

You can still choose to view or change network configuration settings even if the connectivity checks have passed.

However, if you choose to interact with the agent console application rather than letting it time out, you must manually quit the TUI to proceed with the installation.

3. If the agent console application checks have failed, which is indicated by a red icon beside the **Release image URL** pull check, use the following steps to reconfigure the host's network settings:
 - a. Read the **Check Errors** section of the TUI. This section displays error messages specific to the failed checks.



- b. Select **Configure network** to launch the NetworkManager TUI.
- c. Select **Edit a connection** and select the connection you want to reconfigure.
- d. Edit the configuration and select **OK** to save your changes.
- e. Select **Back** to return to the main screen of the NetworkManager TUI.
- f. Select **Activate a Connection**.
- g. Select the reconfigured network to deactivate it.
- h. Select the reconfigured network again to reactivate it.
- i. Select **Back** and then select **Quit** to return to the agent console application.

- j. Wait at least five seconds for the continuous network checks to restart using the new network configuration.
- k. If the **Release image URL** pull check succeeds and displays a green icon beside the URL, select **Quit** to exit the agent console application and continue with the installation.

3.2.5. Tracking and verifying installation progress

Use the following procedure to track installation progress and to verify a successful installation.

Prerequisites

- You have configured a DNS record for the Kubernetes API server.

Procedure

1. Optional: To know when the bootstrap host (rendezvous host) reboots, run the following command:

```
$ ./openshift-install --dir <install_directory> agent wait-for bootstrap-complete \1
--log-level=info 2
```

- 1 For **<install_directory>**, specify the path to the directory where the agent ISO was generated.
- 2 To view different installation details, specify **warn**, **debug**, or **error** instead of **info**.

Example output

```
.....
.....
INFO Bootstrap configMap status is complete
INFO cluster bootstrap is complete
```

The command succeeds when the Kubernetes API server signals that it has been bootstrapped on the control plane machines.

2. To track the progress and verify successful installation, run the following command:

```
$ openshift-install --dir <install_directory> agent wait-for install-complete 1
```

- 1 For **<install_directory>** directory, specify the path to the directory where the agent ISO was generated.

Example output

```
.....
.....
INFO Cluster is installed
INFO Install complete!
INFO To access the cluster as the system:admin user when using 'oc', run
```

```
INFO export KUBECONFIG=/home/core/installer/auth/kubeconfig
INFO Access the OpenShift web-console here: https://console-openshift-console.apps.sno-cluster.test.example.com
```

3.3. GATHERING LOG DATA FROM A FAILED AGENT-BASED INSTALLATION

Use the following procedure to gather log data about a failed Agent-based installation to provide for a support case.

Prerequisites

- You have configured a DNS record for the Kubernetes API server.

Procedure

1. Run the following command and collect the output:

```
$ ./openshift-install --dir <installation_directory> agent wait-for bootstrap-complete --log-level=debug
```

Example error message

```
...
ERROR Bootstrap failed to complete: : bootstrap process timed out: context deadline exceeded
```

2. If the output from the previous command indicates a failure, or if the bootstrap is not progressing, run the following command to connect to the rendezvous host and collect the output:

```
$ ssh core@<node-ip> agent-gather -O >agent-gather.tar.xz
```



NOTE

Red Hat Support can diagnose most issues using the data gathered from the rendezvous host, but if some hosts are not able to register, gathering this data from every host might be helpful.

3. If the bootstrap completes and the cluster nodes reboot, run the following command and collect the output:

```
$ ./openshift-install --dir <install_directory> agent wait-for install-complete --log-level=debug
```

4. If the output from the previous command indicates a failure, perform the following steps:
 - a. Export the **kubeconfig** file to your environment by running the following command:

```
$ export KUBECONFIG=<install_directory>/auth/kubeconfig
```

- b. Gather information for debugging by running the following command:

■

```
$ oc adm must-gather
```

- c. Create a compressed file from the **must-gather** directory that was just created in your working directory by running the following command:

```
$ tar cvaf must-gather.tar.gz <must_gather_directory>
```

5. Excluding the **/auth** subdirectory, attach the installation directory used during the deployment to your support case on the [Red Hat Customer Portal](#).
6. Attach all other data gathered from this procedure to your support case.

CHAPTER 4. INSTALLING A CLUSTER WITH CUSTOMIZATIONS

Use the following procedures to install an OpenShift Container Platform cluster with customizations using the Agent-based Installer.

4.1. PREREQUISITES

- You reviewed details about the [OpenShift Container Platform installation and update](#) processes.
- You read the documentation on [selecting a cluster installation method and preparing it for users](#).
- If you use a firewall or proxy, you [configured it to allow the sites](#) that your cluster requires access to.

4.2. INSTALLING OPENSIFT CONTAINER PLATFORM WITH THE AGENT-BASED INSTALLER

The following procedures deploy a single-node OpenShift Container Platform in a disconnected environment. You can use these procedures as a basis and modify according to your requirements.

4.2.1. Downloading the Agent-based Installer

Use this procedure to download the Agent-based Installer and the CLI needed for your installation.

Procedure

1. Log in to the Red Hat Hybrid Cloud Console using your login credentials.
2. Navigate to [Datacenter](#).
3. Click **Run Agent-based Installer locally**.
4. Select the operating system and architecture for the **OpenShift Installer** and **Command line interface**.
5. Click **Download Installer** to download and extract the install program.
6. Download or copy the pull secret by clicking on **Download pull secret** or **Copy pull secret**.
7. Click **Download command-line tools** and place the **openshift-install** binary in a directory that is on your **PATH**.

4.2.2. Creating the preferred configuration inputs

Use this procedure to create the preferred configuration inputs used to create the agent image.

Procedure

1. Install **nmstate** dependency by running the following command:

```
$ sudo dnf install /usr/bin/nmstatectl -y
```

2. Place the **openshift-install** binary in a directory that is on your PATH.
3. Create a directory to store the install configuration by running the following command:

```
$ mkdir ~/<directory_name>
```



NOTE

This is the preferred method for the Agent-based installation. Using GitOps ZTP manifests is optional.

4. Create the **install-config.yaml** file by running the following command:

```
$ cat << EOF > ./<directory_name>/install-config.yaml
apiVersion: v1
baseDomain: test.example.com
compute:
- architecture: amd64 ❶
  hyperthreading: Enabled
  name: worker
  replicas: 0
controlPlane:
  architecture: amd64
  hyperthreading: Enabled
  name: master
  replicas: 1
metadata:
  name: sno-cluster ❷
networking:
  clusterNetwork:
  - cidr: 10.128.0.0/14
    hostPrefix: 23
  machineNetwork:
  - cidr: 192.168.0.0/16
  networkType: OVNKubernetes ❸
  serviceNetwork:
  - 172.30.0.0/16
platform:
  none: {}
pullSecret: '<pull_secret>' ❹
sshKey: '<ssh_pub_key>' ❺
EOF
```

- ❶ Specify the system architecture, valid values are **amd64** and **arm64**.
- ❷ Required. Specify your cluster name.
- ❸ Specify the cluster network plugin to install. The supported values are **OVNKubernetes** and **OpenShiftSDN**. The default value is **OVNKubernetes**.
- ❹ Specify your pull secret.

5 Specify your SSH public key.



NOTE

If you set the platform to **vSphere**, **baremetal**, or **none**, you can configure IP address endpoints for cluster nodes in three ways:

- IPv4
- IPv6
- IPv4 and IPv6 in parallel (dual-stack)

Example of dual-stack networking

```
networking:
  clusterNetwork:
    - cidr: 172.21.0.0/16
      hostPrefix: 23
    - cidr: fd02::/48
      hostPrefix: 64
  machineNetwork:
    - cidr: 192.168.11.0/16
    - cidr: 2001:DB8::/32
  serviceNetwork:
    - 172.22.0.0/16
    - fd03::/112
  networkType: OVNKubernetes
platform:
  baremetal:
    apiVIPs:
      - 192.168.11.3
      - 2001:DB8::4
    ingressVIPs:
      - 192.168.11.4
      - 2001:DB8::5
```



NOTE

When you use a disconnected mirror registry, you must add the certificate file that you created previously for your mirror registry to the **additionalTrustBundle** field of the **install-config.yaml** file.

5. Create the **agent-config.yaml** file by running the following command:

```
$ cat > agent-config.yaml << EOF
apiVersion: v1beta1
kind: AgentConfig
metadata:
  name: sno-cluster
rendezvousIP: 192.168.111.80 1
hosts: 2
```

```

- hostname: master-0 ❸
  interfaces:
    - name: eno1
      macAddress: 00:ef:44:21:e6:a5
  rootDeviceHints: ❹
    deviceName: /dev/sdb
  networkConfig: ❺
    interfaces:
      - name: eno1
        type: ethernet
        state: up
        mac-address: 00:ef:44:21:e6:a5
        ipv4:
          enabled: true
          address:
            - ip: 192.168.111.80
              prefix-length: 23
          dhcp: false
    dns-resolver:
      config:
        server:
          - 192.168.111.1
    routes:
      config:
        - destination: 0.0.0.0/0
          next-hop-address: 192.168.111.2
          next-hop-interface: eno1
          table-id: 254
EOF

```

- ❶ This IP address is used to determine which node performs the bootstrapping process as well as running the **assisted-service** component. You must provide the rendezvous IP address when you do not specify at least one host's IP address in the **networkConfig** parameter. If this address is not provided, one IP address is selected from the provided hosts' **networkConfig**.
- ❷ Optional: Host configuration. The number of hosts defined must not exceed the total number of hosts defined in the **install-config.yaml** file, which is the sum of the values of the **compute.replicas** and **controlPlane.replicas** parameters.
- ❸ Optional: Overrides the hostname obtained from either the Dynamic Host Configuration Protocol (DHCP) or a reverse DNS lookup. Each host must have a unique hostname supplied by one of these methods.
- ❹ Enables provisioning of the Red Hat Enterprise Linux CoreOS (RHCOS) image to a particular device. The installation program examines the devices in the order it discovers them, and compares the discovered values with the hint values. It uses the first discovered device that matches the hint value.
- ❺ Optional: Configures the network interface of a host in NMState format.

Additional resources

- [Configuring the Agent-based Installer to use mirrored images](#)

4.2.3. Creating additional manifest files

As an optional task, you can create additional manifests to further configure your cluster beyond the configurations available in the **install-config.yaml** and **agent-config.yaml** files.

4.2.3.1. Creating a directory to contain additional manifests

If you create additional manifests to configure your Agent-based installation beyond the **install-config.yaml** and **agent-config.yaml** files, you must create an **openshift** subdirectory within your installation directory. All of your additional machine configurations must be located within this subdirectory.



NOTE

The most common type of additional manifest you can add is a **MachineConfig** object. For examples of **MachineConfig** objects you can add during the Agent-based installation, see "Using MachineConfig objects to configure nodes" in the "Additional resources" section.

Procedure

- On your installation host, create an **openshift** subdirectory within the installation directory by running the following command:

```
$ mkdir <installation_directory>/openshift
```

Additional resources

- [Using MachineConfig objects to configure nodes](#)

4.2.3.2. Disk partitioning

In general, you should use the default disk partitioning that is created during the RHCOS installation. However, there are cases where you might want to create a separate partition for a directory that you expect to grow.

OpenShift Container Platform supports the addition of a single partition to attach storage to either the **/var** directory or a subdirectory of **/var**. For example:

- /var/lib/containers:** Holds container-related content that can grow as more images and containers are added to a system.
- /var/lib/etcd:** Holds data that you might want to keep separate for purposes such as performance optimization of etcd storage.
- /var:** Holds data that you might want to keep separate for purposes such as auditing.



IMPORTANT

For disk sizes larger than 100GB, and especially larger than 1TB, create a separate **/var** partition.

Storing the contents of a **/var** directory separately makes it easier to grow storage for those areas as

needed and reinstall OpenShift Container Platform at a later date and keep that data intact. With this method, you will not have to pull all your containers again, nor will you have to copy massive log files when you update systems.

The use of a separate partition for the **/var** directory or a subdirectory of **/var** also prevents data growth in the partitioned directory from filling up the root file system.

The following procedure sets up a separate **/var** partition by adding a machine config manifest that is wrapped into the Ignition config file for a node type during the preparation phase of an installation.

Prerequisites

- You have created an **openshift** subdirectory within your installation directory.

Procedure

1. Create a Butane config that configures the additional partition. For example, name the file **\$HOME/clusterconfig/98-var-partition.bu**, change the disk device name to the name of the storage device on the **worker** systems, and set the storage size as appropriate. This example places the **/var** directory on a separate partition:

```
variant: openshift
version: 4.14.0
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: 98-var-partition
storage:
  disks:
    - device: /dev/disk/by-id/<device_name> ❶
  partitions:
    - label: var
      start_mib: <partition_start_offset> ❷
      size_mib: <partition_size> ❸
      number: 5
  filesystems:
    - device: /dev/disk/by-partlabel/var
      path: /var
      format: xfs
      mount_options: [defaults, prjquota] ❹
      with_mount_unit: true
```

- ❶ The storage device name of the disk that you want to partition.
- ❷ When adding a data partition to the boot disk, a minimum offset value of 25000 mebibytes is recommended. The root file system is automatically resized to fill all available space up to the specified offset. If no offset value is specified, or if the specified value is smaller than the recommended minimum, the resulting root file system will be too small, and future reinstalls of RHCOS might overwrite the beginning of the data partition.
- ❸ The size of the data partition in mebibytes.
- ❹ The **prjquota** mount option must be enabled for filesystems used for container storage.

**NOTE**

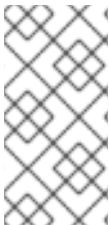
When creating a separate **/var** partition, you cannot use different instance types for compute nodes, if the different instance types do not have the same device name.

2. Create a manifest from the Butane config and save it to the **clusterconfig/openshift** directory. For example, run the following command:

```
$ butane $HOME/clusterconfig/98-var-partition.bu -o $HOME/clusterconfig/openshift/98-var-partition.yaml
```

4.2.4. Using ZTP manifests

As an optional task, you can use GitOps Zero Touch Provisioning (ZTP) manifests to configure your installation beyond the options available through the **install-config.yaml** and **agent-config.yaml** files.

**NOTE**

GitOps ZTP manifests can be generated with or without configuring the **install-config.yaml** and **agent-config.yaml** files beforehand. If you chose to configure the **install-config.yaml** and **agent-config.yaml** files, the configurations will be imported to the ZTP cluster manifests when they are generated.

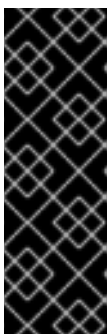
Prerequisites

- You have placed the **openshift-install** binary in a directory that is on your **PATH**.
- Optional: You have created and configured the **install-config.yaml** and **agent-config.yaml** files.

Procedure

1. Generate ZTP cluster manifests by running the following command:

```
$ openshift-install agent create cluster-manifests --dir <installation_directory>
```

**IMPORTANT**

If you have created the **install-config.yaml** and **agent-config.yaml** files, those files are deleted and replaced by the cluster manifests generated through this command.

Any configurations made to the **install-config.yaml** and **agent-config.yaml** files are imported to the ZTP cluster manifests when you run the **openshift-install agent create cluster-manifests** command.

2. Navigate to the **cluster-manifests** directory by running the following command:

```
$ cd <installation_directory>/cluster-manifests
```

3. Configure the manifest files in the **cluster-manifests** directory. For sample files, see the "Sample GitOps ZTP custom resources" section.
4. Disconnected clusters: If you did not define mirror configuration in the **install-config.yaml** file before generating the ZTP manifests, perform the following steps:
 - a. Navigate to the **mirror** directory by running the following command:

```
$ cd ../mirror
```

- b. Configure the manifest files in the **mirror** directory.

Additional resources

- [Sample GitOps ZTP custom resources](#) .
- See [Challenges of the network far edge](#) to learn more about GitOps Zero Touch Provisioning (ZTP).

4.2.5. Encrypting the disk

As an optional task, you can use this procedure to encrypt your disk or partition while installing OpenShift Container Platform with the Agent-based Installer.



IMPORTANT

If there are leftover TPM encryption keys from a previous operating system on the bare-metal host, the cluster deployment can get stuck. To avoid this situation, it is highly recommended to reset the TPM chip in the BIOS before booting the ISO.

Prerequisites

- You have created and configured the **install-config.yaml** and **agent-config.yaml** files, unless you are using ZTP manifests.
- You have placed the **openshift-install** binary in a directory that is on your **PATH**.

Procedure

1. Generate ZTP cluster manifests by running the following command:

```
$ openshift-install agent create cluster-manifests --dir <installation_directory>
```



IMPORTANT

If you have created the **install-config.yaml** and **agent-config.yaml** files, those files are deleted and replaced by the cluster manifests generated through this command.

Any configurations made to the **install-config.yaml** and **agent-config.yaml** files are imported to the ZTP cluster manifests when you run the **openshift-install agent create cluster-manifests** command.

**NOTE**

If you have already generated ZTP manifests, skip this step.

2. Navigate to the **cluster-manifests** directory by running the following command:

```
$ cd <installation_directory>/cluster-manifests
```

3. Add the following section to the **agent-cluster-install.yaml** file:

```
diskEncryption:
  enableOn: all 1
  mode: tang 2
  tangServers: "server1": "http://tang-server-1.example.com:7500" 3
```

- 1** Specify which nodes to enable disk encryption on. Valid values are **none**, **all**, **masters**, and **workers**.
- 2** Specify which disk encryption mode to use. Valid values are **tpmv2** and **tang**.
- 3** Optional: If you are using Tang, specify the Tang servers.

Additional resources

- [About disk encryption](#)

4.2.6. Creating and booting the agent image

Use this procedure to boot the agent image on your machines.

Procedure

1. Create the agent image by running the following command:

```
$ openshift-install --dir <install_directory> agent create image
```

**NOTE**

Red Hat Enterprise Linux CoreOS (RHCOS) supports multipathing on the primary disk, allowing stronger resilience to hardware failure to achieve higher host availability. Multipathing is enabled by default in the agent ISO image, with a default **/etc/multipath.conf** configuration.

2. Boot the **agent.x86_64.iso** or **agent.aarch64.iso** image on the bare metal machines.

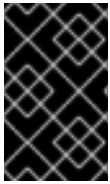
4.2.7. Verifying that the current installation host can pull release images

After you boot the agent image and network services are made available to the host, the agent console application performs a pull check to verify that the current host can retrieve release images.

If the primary pull check passes, you can quit the application to continue with the installation. If the pull

check fails, the application performs additional checks, as seen in the **Additional checks** section of the TUI, to help you troubleshoot the problem. A failure for any of the additional checks is not necessarily critical as long as the primary pull check succeeds.

If there are host network configuration issues that might cause an installation to fail, you can use the console application to make adjustments to your network configurations.



IMPORTANT

If the agent console application detects host network configuration issues, the installation workflow will be halted until the user manually stops the console application and signals the intention to proceed.

Procedure

1. Wait for the agent console application to check whether or not the configured release image can be pulled from a registry.
2. If the agent console application states that the installer connectivity checks have passed, wait for the prompt to time out to continue with the installation.

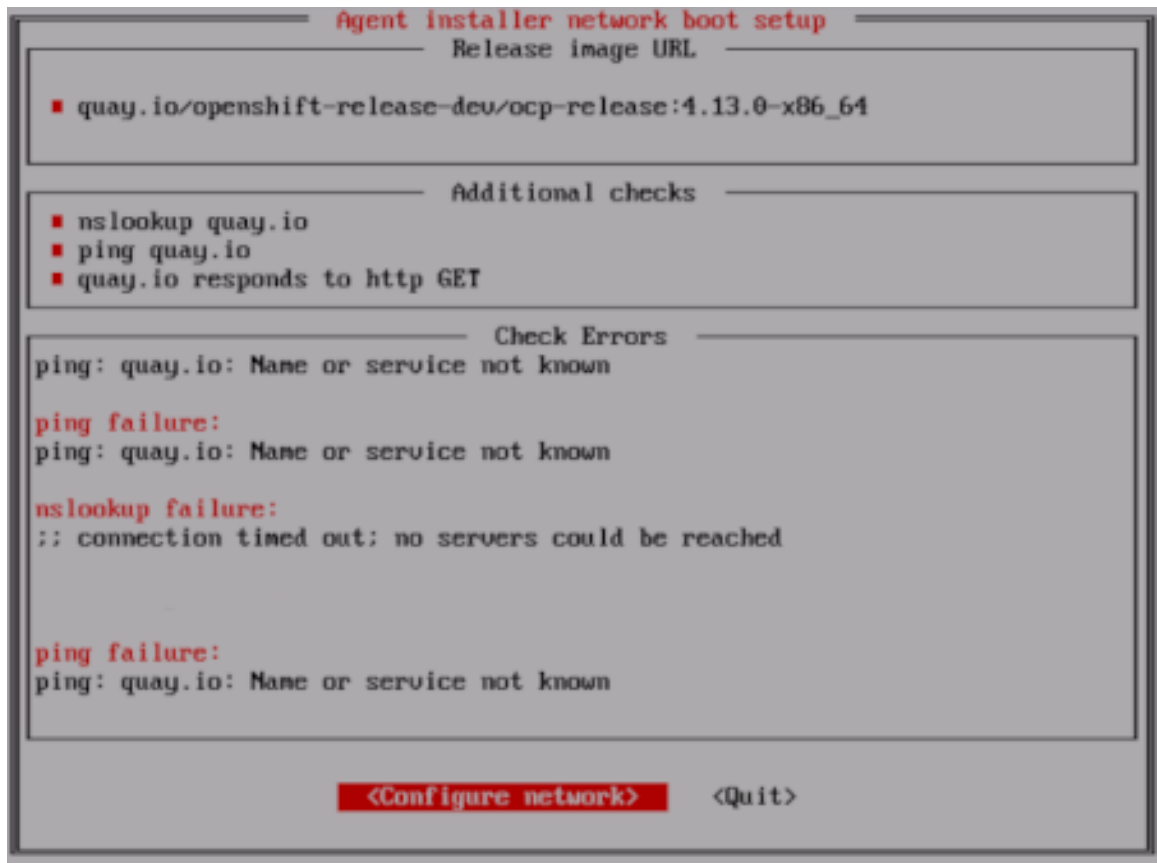


NOTE

You can still choose to view or change network configuration settings even if the connectivity checks have passed.

However, if you choose to interact with the agent console application rather than letting it time out, you must manually quit the TUI to proceed with the installation.

3. If the agent console application checks have failed, which is indicated by a red icon beside the **Release image URL** pull check, use the following steps to reconfigure the host's network settings:
 - a. Read the **Check Errors** section of the TUI. This section displays error messages specific to the failed checks.



- b. Select **Configure network** to launch the NetworkManager TUI.
- c. Select **Edit a connection** and select the connection you want to reconfigure.
- d. Edit the configuration and select **OK** to save your changes.
- e. Select **Back** to return to the main screen of the NetworkManager TUI.
- f. Select **Activate a Connection**.
- g. Select the reconfigured network to deactivate it.
- h. Select the reconfigured network again to reactivate it.
- i. Select **Back** and then select **Quit** to return to the agent console application.
- j. Wait at least five seconds for the continuous network checks to restart using the new network configuration.
- k. If the **Release image URL** pull check succeeds and displays a green icon beside the URL, select **Quit** to exit the agent console application and continue with the installation.

4.2.8. Tracking and verifying installation progress

Use the following procedure to track installation progress and to verify a successful installation.

Prerequisites

- You have configured a DNS record for the Kubernetes API server.

Procedure

- Optional: To know when the bootstrap host (rendezvous host) reboots, run the following command:

```
$ ./openshift-install --dir <install_directory> agent wait-for bootstrap-complete \1
--log-level=info 2
```

- For **<install_directory>**, specify the path to the directory where the agent ISO was generated.
- To view different installation details, specify **warn**, **debug**, or **error** instead of **info**.

Example output

```
.....
.....
INFO Bootstrap configMap status is complete
INFO cluster bootstrap is complete
```

The command succeeds when the Kubernetes API server signals that it has been bootstrapped on the control plane machines.

- To track the progress and verify successful installation, run the following command:

```
$ openshift-install --dir <install_directory> agent wait-for install-complete 1
```

- For **<install_directory>** directory, specify the path to the directory where the agent ISO was generated.

Example output

```
.....
.....
INFO Cluster is installed
INFO Install complete!
INFO To access the cluster as the system:admin user when using 'oc', run
INFO   export KUBECONFIG=/home/core/installer/auth/kubeconfig
INFO Access the OpenShift web-console here: https://console-openshift-console.apps.sno-
cluster.test.example.com
```

NOTE

If you are using the optional method of GitOps ZTP manifests, you can configure IP address endpoints for cluster nodes through the **AgentClusterInstall.yaml** file in three ways:

- IPv4
- IPv6
- IPv4 and IPv6 in parallel (dual-stack)

IPv6 is supported only on bare metal platforms.

Example of dual-stack networking

```
apiVIP: 192.168.11.3
ingressVIP: 192.168.11.4
clusterDeploymentRef:
  name: mycluster
imageSetRef:
  name: openshift-4.14
networking:
  clusterNetwork:
    - cidr: 172.21.0.0/16
      hostPrefix: 23
    - cidr: fd02::/48
      hostPrefix: 64
  machineNetwork:
    - cidr: 192.168.11.0/16
    - cidr: 2001:DB8::/32
  serviceNetwork:
    - 172.22.0.0/16
    - fd03::/112
  networkType: OVNKubernetes
```

Additional resources

- See [Deploying with dual-stack networking](#).
- See [Configuring the install-config yaml file](#).
- See [Configuring a three-node cluster](#) to deploy three-node clusters in bare metal environments.
- See [About root device hints](#).
- See [NMState state examples](#).

4.3. SAMPLE GITOPS ZTP CUSTOM RESOURCES

You can optionally use GitOps Zero Touch Provisioning (ZTP) custom resource (CR) objects to install an OpenShift Container Platform cluster with the Agent-based Installer.

You can customize the following GitOps ZTP custom resources to specify more details about your OpenShift Container Platform cluster. The following sample GitOps ZTP custom resources are for a single-node cluster.

Example agent-cluster-install.yaml file

```
apiVersion: extensions.hive.openshift.io/v1beta1
kind: AgentClusterInstall
metadata:
  name: test-agent-cluster-install
  namespace: cluster0
spec:
  clusterDeploymentRef:
    name: otest
  imageSetRef:
```



```

    name: openshift-4.14
networking:
  clusterNetwork:
    - cidr: 10.128.0.0/14
      hostPrefix: 23
  serviceNetwork:
    - 172.30.0.0/16
provisionRequirements:
  controlPlaneAgents: 1
  workerAgents: 0
sshPublicKey: <ssh_public_key>

```

Example cluster-deployment.yaml file

```

apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: otest
  namespace: cluster0
spec:
  baseDomain: test.metalkube.org
  clusterInstallRef:
    group: extensions.hive.openshift.io
    kind: AgentClusterInstall
    name: test-agent-cluster-install
    version: v1beta1
  clusterName: otest
  controlPlaneConfig:
    servingCertificates: {}
  platform:
    agentBareMetal:
      agentSelector:
        matchLabels:
          bla: aaa
  pullSecretRef:
    name: pull-secret

```

Example cluster-image-set.yaml file

```

apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  name: openshift-4.14
spec:
  releasImage: registry.ci.openshift.org/ocp/release:4.14.0-0.nightly-2022-06-06-025509

```

Example infra-env.yaml file

```

apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: myinfraenv
  namespace: cluster0
spec:

```

```

clusterRef:
  name: otest
  namespace: cluster0
cpuArchitecture: aarch64
pullSecretRef:
  name: pull-secret
sshAuthorizedKey: <ssh_public_key>
nmStateConfigLabelSelector:
  matchLabels:
    cluster0-nmstate-label-name: cluster0-nmstate-label-value

```

Example nmstateconfig.yaml file

```

apiVersion: agent-install.openshift.io/v1beta1
kind: NMStateConfig
metadata:
  name: master-0
  namespace: openshift-machine-api
  labels:
    cluster0-nmstate-label-name: cluster0-nmstate-label-value
spec:
  config:
    interfaces:
      - name: eth0
        type: ethernet
        state: up
        mac-address: 52:54:01:aa:aa:a1
        ipv4:
          enabled: true
          address:
            - ip: 192.168.122.2
              prefix-length: 23
          dhcp: false
        dns-resolver:
          config:
            server:
              - 192.168.122.1
        routes:
          config:
            - destination: 0.0.0.0/0
              next-hop-address: 192.168.122.1
              next-hop-interface: eth0
              table-id: 254
    interfaces:
      - name: "eth0"
        macAddress: 52:54:01:aa:aa:a1

```

Example pull-secret.yaml file

```

apiVersion: v1
kind: Secret
type: kubernetes.io/dockerconfigjson
metadata:
  name: pull-secret

```

```
namespace: cluster0
stringData:
  .dockerconfigjson: <pull_secret>
```

Additional resources

- See [Challenges of the network far edge](#) to learn more about GitOps Zero Touch Provisioning (ZTP).

4.4. GATHERING LOG DATA FROM A FAILED AGENT-BASED INSTALLATION

Use the following procedure to gather log data about a failed Agent-based installation to provide for a support case.

Prerequisites

- You have configured a DNS record for the Kubernetes API server.

Procedure

1. Run the following command and collect the output:

```
$ ./openshift-install --dir <installation_directory> agent wait-for bootstrap-complete --log-level=debug
```

Example error message

```
...
ERROR Bootstrap failed to complete: : bootstrap process timed out: context deadline exceeded
```

2. If the output from the previous command indicates a failure, or if the bootstrap is not progressing, run the following command to connect to the rendezvous host and collect the output:

```
$ ssh core@<node-ip> agent-gather -O >agent-gather.tar.xz
```



NOTE

Red Hat Support can diagnose most issues using the data gathered from the rendezvous host, but if some hosts are not able to register, gathering this data from every host might be helpful.

3. If the bootstrap completes and the cluster nodes reboot, run the following command and collect the output:

```
$ ./openshift-install --dir <install_directory> agent wait-for install-complete --log-level=debug
```

4. If the output from the previous command indicates a failure, perform the following steps:
 - a. Export the **kubeconfig** file to your environment by running the following command:

```
$ export KUBECONFIG=<install_directory>/auth/kubeconfig
```

- b. Gather information for debugging by running the following command:

```
$ oc adm must-gather
```

- c. Create a compressed file from the **must-gather** directory that was just created in your working directory by running the following command:

```
$ tar cvaf must-gather.tar.gz <must_gather_directory>
```

5. Excluding the **/auth** subdirectory, attach the installation directory used during the deployment to your support case on the [Red Hat Customer Portal](#).
6. Attach all other data gathered from this procedure to your support case.

CHAPTER 5. PREPARING PXE ASSETS FOR OPENSIFT CONTAINER PLATFORM

Use the following procedures to create the assets needed to PXE boot an OpenShift Container Platform cluster using the Agent-based Installer.

The assets you create in these procedures will deploy a single-node OpenShift Container Platform installation. You can use these procedures as a basis and modify configurations according to your requirements.

See [Installing an OpenShift Container Platform cluster with the Agent-based Installer](#) to learn about more configurations available with the Agent-based Installer.

5.1. PREREQUISITES

- You reviewed details about the [OpenShift Container Platform installation and update](#) processes.

5.2. DOWNLOADING THE AGENT-BASED INSTALLER

Use this procedure to download the Agent-based Installer and the CLI needed for your installation.

Procedure

1. Log in to the Red Hat Hybrid Cloud Console using your login credentials.
2. Navigate to [Datacenter](#).
3. Click **Run Agent-based Installer locally**.
4. Select the operating system and architecture for the **OpenShift Installer** and **Command line interface**.
5. Click **Download Installer** to download and extract the install program.
6. Download or copy the pull secret by clicking on **Download pull secret** or **Copy pull secret**.
7. Click **Download command-line tools** and place the **openshift-install** binary in a directory that is on your **PATH**.

5.3. CREATING THE PREFERRED CONFIGURATION INPUTS

Use this procedure to create the preferred configuration inputs used to create the PXE files.

Procedure

1. Install **nmstate** dependency by running the following command:

```
$ sudo dnf install /usr/bin/nmstatectl -y
```

2. Place the **openshift-install** binary in a directory that is on your PATH.
3. Create a directory to store the install configuration by running the following command:

```
$ mkdir ~/<directory_name>
```



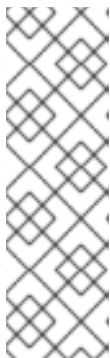
NOTE

This is the preferred method for the Agent-based installation. Using GitOps ZTP manifests is optional.

4. Create the **install-config.yaml** file by running the following command:

```
$ cat << EOF > ./<directory_name>/install-config.yaml
apiVersion: v1
baseDomain: test.example.com
compute:
- architecture: amd64 1
  hyperthreading: Enabled
  name: worker
  replicas: 0
controlPlane:
  architecture: amd64
  hyperthreading: Enabled
  name: master
  replicas: 1
metadata:
  name: sno-cluster 2
networking:
  clusterNetwork:
  - cidr: 10.128.0.0/14
    hostPrefix: 23
  machineNetwork:
  - cidr: 192.168.0.0/16
    networkType: OVNKubernetes 3
  serviceNetwork:
  - 172.30.0.0/16
platform:
  none: {}
pullSecret: '<pull_secret>' 4
sshKey: '<ssh_pub_key>' 5
EOF
```

- 1** Specify the system architecture, valid values are **amd64** and **arm64**.
- 2** Required. Specify your cluster name.
- 3** Specify the cluster network plugin to install. The supported values are **OVNKubernetes** and **OpenShiftSDN**. The default value is **OVNKubernetes**.
- 4** Specify your pull secret.
- 5** Specify your SSH public key.



NOTE

If you set the platform to **vSphere**, **baremetal**, or **none**, you can configure IP address endpoints for cluster nodes in three ways:

- IPv4
- IPv6
- IPv4 and IPv6 in parallel (dual-stack)

Example of dual-stack networking

```
networking:
  clusterNetwork:
    - cidr: 172.21.0.0/16
      hostPrefix: 23
    - cidr: fd02::/48
      hostPrefix: 64
  machineNetwork:
    - cidr: 192.168.11.0/16
    - cidr: 2001:DB8::/32
  serviceNetwork:
    - 172.22.0.0/16
    - fd03::/112
  networkType: OVNKubernetes
platform:
  baremetal:
    apiVIPs:
      - 192.168.11.3
      - 2001:DB8::4
    ingressVIPs:
      - 192.168.11.4
      - 2001:DB8::5
```



NOTE

When you use a disconnected mirror registry, you must add the certificate file that you created previously for your mirror registry to the **additionalTrustBundle** field of the **install-config.yaml** file.

5. Create the **agent-config.yaml** file by running the following command:

```
$ cat > agent-config.yaml << EOF
apiVersion: v1beta1
kind: AgentConfig
metadata:
  name: sno-cluster
rendezvousIP: 192.168.111.80 ❶
hosts: ❷
  - hostname: master-0 ❸
    interfaces:
      - name: eno1
        macAddress: 00:ef:44:21:e6:a5
```

```

rootDeviceHints: 4
  deviceName: /dev/sdb
networkConfig: 5
  interfaces:
    - name: eno1
      type: ethernet
      state: up
      mac-address: 00:ef:44:21:e6:a5
      ipv4:
        enabled: true
        address:
          - ip: 192.168.111.80
            prefix-length: 23
        dhcp: false
  dns-resolver:
    config:
      server:
        - 192.168.111.1
  routes:
    config:
      - destination: 0.0.0.0/0
        next-hop-address: 192.168.111.2
        next-hop-interface: eno1
        table-id: 254
EOF

```

- 1 This IP address is used to determine which node performs the bootstrapping process as well as running the **assisted-service** component. You must provide the rendezvous IP address when you do not specify at least one host's IP address in the **networkConfig** parameter. If this address is not provided, one IP address is selected from the provided hosts' **networkConfig**.
 - 2 Optional: Host configuration. The number of hosts defined must not exceed the total number of hosts defined in the **install-config.yaml** file, which is the sum of the values of the **compute.replicas** and **controlPlane.replicas** parameters.
 - 3 Optional: Overrides the hostname obtained from either the Dynamic Host Configuration Protocol (DHCP) or a reverse DNS lookup. Each host must have a unique hostname supplied by one of these methods.
 - 4 Enables provisioning of the Red Hat Enterprise Linux CoreOS (RHCOS) image to a particular device. The installation program examines the devices in the order it discovers them, and compares the discovered values with the hint values. It uses the first discovered device that matches the hint value.
 - 5 Optional: Configures the network interface of a host in NMState format.
6. Optional: To create an iPXE script, add the **bootArtifactsBaseURL** to the **agent-config.yaml** file:

```

apiVersion: v1beta1
kind: AgentConfig
metadata:

```



```
name: sno-cluster
rendezvousIP: 192.168.111.80
bootArtifactsBaseURL: <asset_server_URL>
```

Where **<asset_server_URL>** is the URL of the server you will upload the PXE assets to.

Additional resources

- [Deploying with dual-stack networking](#).
- [Configuring the install-config.yaml file](#).
- See [Configuring a three-node cluster](#) to deploy three-node clusters in bare metal environments.
- [About root device hints](#).
- [NMState state examples](#).
- [Optional: Creating additional manifest files](#)

5.4. CREATING THE PXE ASSETS

Use the following procedure to create the assets and optional script to implement in your PXE infrastructure.

Procedure

1. Create the PXE assets by running the following command:

```
$ openshift-install agent create pxe-files
```

The generated PXE assets and optional iPXE script can be found in the **boot-artifacts** directory.

Example filesystem with PXE assets and optional iPXE script

```
boot-artifacts
├── agent.x86_64-initrd.img
├── agent.x86_64.ipxe
├── agent.x86_64-rootfs.img
└── agent.x86_64-vmlinuz
```



NOTE

Red Hat Enterprise Linux CoreOS (RHCOS) supports multipathing on the primary disk, allowing stronger resilience to hardware failure to achieve higher host availability. Multipathing is enabled by default in the agent ISO image, with a default **/etc/multipath.conf** configuration.

2. Upload the PXE assets and optional script to your infrastructure where they will be accessible during the boot process.

**NOTE**

If you generated an iPXE script, the location of the assets must match the **bootArtifactsBaseURL** you added to the **agent-config.yaml** file.

CHAPTER 6. PREPARING AN AGENT-BASED INSTALLED CLUSTER FOR THE MULTICLUSTER ENGINE FOR KUBERNETES OPERATOR

You can install the multicluster engine Operator and deploy a hub cluster with the Agent-based OpenShift Container Platform Installer. The following procedure is partially automated and requires manual steps after the initial cluster is deployed.

6.1. PREREQUISITES

- You have read the following documentation:
 - [Cluster lifecycle with multicluster engine operator overview](#) .
 - [Persistent storage using local volumes](#) .
 - [Preparing to install with the Agent-based Installer](#) .
 - [About disconnected installation mirroring](#) .
- You have access to the internet to obtain the necessary container images.
- You have installed the OpenShift CLI (**oc**).
- If you are installing in a disconnected environment, you must have a configured local mirror registry for disconnected installation mirroring.

6.2. PREPARING AN AGENT-BASED CLUSTER DEPLOYMENT FOR THE MULTICLUSTER ENGINE FOR KUBERNETES OPERATOR WHILE DISCONNECTED

You can mirror the required OpenShift Container Platform container images, the multicluster engine Operator, and the Local Storage Operator (LSO) into your local mirror registry in a disconnected environment. Ensure that you note the local DNS hostname and port of your mirror registry.



NOTE

To mirror your OpenShift Container Platform image repository to your mirror registry, you can use either the **oc adm release image** or **oc mirror** command. In this procedure, the **oc mirror** command is used as an example.

Procedure

1. Create an **<assets_directory>** folder to contain valid **install-config.yaml** and **agent-config.yaml** files. This directory is used to store all the assets.
2. To mirror an OpenShift Container Platform image repository, the multicluster engine, and the LSO, create a **ImageSetConfiguration.yaml** file with the following settings:

Example ImageSetConfiguration.yaml

```
kind: ImageSetConfiguration
```

```

apiVersion: mirror.openshift.io/v1alpha2
archiveSize: 4 1
storageConfig: 2
  imageURL: <your-local-registry-dns-name>:<your-local-registry-port>/mirror/oc-mirror-
metadata 3
  skipTLS: true
mirror:
  platform:
    architectures:
      - "amd64"
    channels:
      - name: stable-4.14 4
        type: ocp
  additionalImages:
    - name: registry.redhat.io/ubi9/ubi:latest
  operators:
    - catalog: registry.redhat.io/redhat/redhat-operator-index:v4.14 5
  packages: 6
    - name: multicluster-engine 7
    - name: local-storage-operator 8

```

- ¹ Specify the maximum size, in GiB, of each file within the image set.
- ² Set the back-end location to receive the image set metadata. This location can be a registry or local directory. It is required to specify **storageConfig** values.
- ³ Set the registry URL for the storage backend.
- ⁴ Set the channel that contains the OpenShift Container Platform images for the version you are installing.
- ⁵ Set the Operator catalog that contains the OpenShift Container Platform images that you are installing.
- ⁶ Specify only certain Operator packages and channels to include in the image set. Remove this field to retrieve all packages in the catalog.
- ⁷ The multicluster engine packages and channels.
- ⁸ The LSO packages and channels.



NOTE

This file is required by the **oc mirror** command when mirroring content.

3. To mirror a specific OpenShift Container Platform image repository, the multicluster engine, and the LSO, run the following command:

```
$ oc mirror --dest-skip-tls --config ocp-mce-imageset.yaml docker://<your-local-registry-dns-name>:<your-local-registry-port>
```

4. Update the registry and certificate in the **install-config.yaml** file:

Example imageContentSources.yaml

```
imageContentSources:
- source: "quay.io/openshift-release-dev/ocp-release"
  mirrors:
    - "<your-local-registry-dns-name>:<your-local-registry-port>/openshift/release-images"
- source: "quay.io/openshift-release-dev/ocp-v4.0-art-dev"
  mirrors:
    - "<your-local-registry-dns-name>:<your-local-registry-port>/openshift/release"
- source: "registry.redhat.io/ubi9"
  mirrors:
    - "<your-local-registry-dns-name>:<your-local-registry-port>/ubi9"
- source: "registry.redhat.io/multicluster-engine"
  mirrors:
    - "<your-local-registry-dns-name>:<your-local-registry-port>/multicluster-engine"
- source: "registry.redhat.io/rhel8"
  mirrors:
    - "<your-local-registry-dns-name>:<your-local-registry-port>/rhel8"
- source: "registry.redhat.io/redhat"
  mirrors:
    - "<your-local-registry-dns-name>:<your-local-registry-port>/redhat"
```

Additionally, ensure your certificate is present in the **additionalTrustBundle** field of the **install-config.yaml**.

Example install-config.yaml

```
additionalTrustBundle: |
-----BEGIN CERTIFICATE-----
ZZZZZZZZZZZZ
-----END CERTIFICATE-----
```



IMPORTANT

The **oc mirror** command creates a folder called **oc-mirror-workspace** with several outputs. This includes the **imageContentSourcePolicy.yaml** file that identifies all the mirrors you need for OpenShift Container Platform and your selected Operators.

5. Generate the cluster manifests by running the following command:

```
$ openshift-install agent create cluster-manifests
```

This command updates the cluster manifests folder to include a **mirror** folder that contains your mirror configuration.

6.3. PREPARING AN AGENT-BASED CLUSTER DEPLOYMENT FOR THE MULTICLUSTER ENGINE FOR KUBERNETES OPERATOR WHILE CONNECTED

Create the required manifests for the multicluster engine Operator, the Local Storage Operator (LSO), and to deploy an agent-based OpenShift Container Platform cluster as a hub cluster.

Procedure

1. Create a sub-folder named **openshift** in the **<assets_directory>** folder. This sub-folder is used to store the extra manifests that will be applied during the installation to further customize the deployed cluster. The **<assets_directory>** folder contains all the assets including the **install-config.yaml** and **agent-config.yaml** files.



NOTE

The installer does not validate extra manifests.

2. For the multicluster engine, create the following manifests and save them in the **<assets_directory>/openshift** folder:

Example mce_namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  labels:
    openshift.io/cluster-monitoring: "true"
  name: multicluster-engine
```

Example mce_operatorgroup.yaml

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: multicluster-engine-operatorgroup
  namespace: multicluster-engine
spec:
  targetNamespaces:
    - multicluster-engine
```

Example mce_subscription.yaml

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: multicluster-engine
  namespace: multicluster-engine
spec:
  channel: "stable-2.3"
  name: multicluster-engine
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```



NOTE

You can install a distributed unit (DU) at scale with the Red Hat Advanced Cluster Management (RHACM) using the assisted installer (AI). These distributed units must be enabled in the hub cluster. The AI service requires persistent volumes (PVs), which are manually created.

- For the AI service, create the following manifests and save them in the **<assets_directory>/openshift** folder:

Example Iso_namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  annotations:
    openshift.io/cluster-monitoring: "true"
  name: openshift-local-storage
```

Example Iso_operatorgroup.yaml

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: local-operator-group
  namespace: openshift-local-storage
spec:
  targetNamespaces:
    - openshift-local-storage
```

Example Iso_subscription.yaml

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: local-storage-operator
  namespace: openshift-local-storage
spec:
  installPlanApproval: Automatic
  name: local-storage-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

NOTE

After creating all the manifests, your filesystem must display as follows:

Example Filesystem

```
<assets_directory>
├── install-config.yaml
├── agent-config.yaml
├── /openshift
│   ├── mce_namespace.yaml
│   ├── mce_operatorgroup.yaml
│   ├── mce_subscription.yaml
│   ├── Iso_namespace.yaml
│   ├── Iso_operatorgroup.yaml
│   └── Iso_subscription.yaml
```

4. Create the agent ISO image by running the following command:

```
$ openshift-install agent create image --dir <assets_directory>
```

5. When the image is ready, boot the target machine and wait for the installation to complete.
6. To monitor the installation, run the following command:

```
$ openshift-install agent wait-for install-complete --dir <assets_directory>
```



NOTE

To configure a fully functional hub cluster, you must create the following manifests and manually apply them by running the command **\$ oc apply -f <manifest-name>**. The order of the manifest creation is important and where required, the waiting condition is displayed.

7. For the PVs that are required by the AI service, create the following manifests:

```
apiVersion: local.storage.openshift.io/v1
kind: LocalVolume
metadata:
  name: assisted-service
  namespace: openshift-local-storage
spec:
  logLevel: Normal
  managementState: Managed
  storageClassDevices:
    - devicePaths:
        - /dev/vda
        - /dev/vdb
      storageClassName: assisted-service
      volumeMode: Filesystem
```

8. Use the following command to wait for the availability of the PVs, before applying the subsequent manifests:

```
$ oc wait localvolume -n openshift-local-storage assisted-service --for condition=Available --
timeout 10m
```



NOTE

The ``devicePath`` is an example and may vary depending on the actual hardware configuration used.

9. Create a manifest for a multicluster engine instance.

Example MultiClusterEngine.yaml

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
```



```
name: multiclusterengine
spec: {}
```

10. Create a manifest to enable the AI service.

Example `agentserviceconfig.yaml`

```
apiVersion: agent-install.openshift.io/v1beta1
kind: AgentServiceConfig
metadata:
  name: agent
  namespace: assisted-installer
spec:
  databaseStorage:
    storageClassName: assisted-service
    accessModes:
      - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  filesystemStorage:
    storageClassName: assisted-service
    accessModes:
      - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

11. Create a manifest to deploy subsequently spoke clusters.

Example `clusterimageset.yaml`

```
apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  name: "4.14"
spec:
  releaseImage: quay.io/openshift-release-dev/ocp-release:4.14.0-x86_64
```

12. Create a manifest to import the agent installed cluster (that hosts the multicluster engine and the Assisted Service) as the hub cluster.

Example `autoimport.yaml`

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  labels:
    local-cluster: "true"
    cloud: auto-detect
    vendor: auto-detect
  name: local-cluster
spec:
  hubAcceptsClient: true
```

13. Wait for the managed cluster to be created.

```
$ oc wait -n multicluster-engine managedclusters local-cluster --for  
condition=ManagedClusterJoined=True --timeout 10m
```

Verification

- To confirm that the managed cluster installation is successful, run the following command:

```
$ oc get managedcluster  
NAME          HUB ACCEPTED  MANAGED CLUSTER URLS      JOINED  AVAILABLE  
AGE  
local-cluster  true          https://<your cluster url>:6443  True    True    77m
```

Additional resources

- [The Local Storage Operator](#)

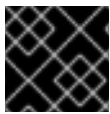
CHAPTER 7. INSTALLATION CONFIGURATION PARAMETERS FOR THE AGENT-BASED INSTALLER

Before you deploy an OpenShift Container Platform cluster using the Agent-based Installer, you provide parameters to customize your cluster and the platform that hosts it. When you create the **install-config.yaml** and **agent-config.yaml** files, you must provide values for the required parameters, and you can use the optional parameters to customize your cluster further.

7.1. AVAILABLE INSTALLATION CONFIGURATION PARAMETERS

The following tables specify the required and optional installation configuration parameters that you can set as part of the Agent-based installation process.

These values are specified in the **install-config.yaml** file.



IMPORTANT

These settings are used for installation only, and cannot be changed after installation.

7.1.1. Required configuration parameters

Required installation configuration parameters are described in the following table:

Table 7.1. Required parameters

Parameter	Description	Values
apiVersion:	The API version for the install-config.yaml content. The current version is v1 . The installation program might also support older API versions.	String
baseDomain:	The base domain of your cloud provider. The base domain is used to create routes to your OpenShift Container Platform cluster components. The full DNS name for your cluster is a combination of the baseDomain and metadata.name parameter values that uses the <metadata.name> . <baseDomain> format.	A fully-qualified domain or subdomain name, such as example.com .

Parameter	Description	Values
metadata:	Kubernetes resource ObjectMeta , from which only the name parameter is consumed.	Object
metadata: name:	The name of the cluster. DNS records for the cluster are all subdomains of {{.metadata.name}}.{{.baseDomain}} . The cluster name is set to agent-cluster when you do not provide the metadata.name parameter through either the install-config.yaml or agent-config.yaml files. For example, installations that only use ZTP manifests do not provide the metadata.name parameter.	String of lowercase letters, hyphens (-), and periods (.), such as dev .
platform:	The configuration for the specific platform upon which to perform the installation: baremetal , external , none , or vsphere .	Object
pullSecret:	Get a pull secret from Red Hat OpenShift Cluster Manager to authenticate downloading container images for OpenShift Container Platform components from services such as Quay.io.	<pre>{ "auths":{ "cloud.openshift.com":{ "auth":"b3Blb=", "email":"you@example.com" }, "quay.io":{ "auth":"b3Blb=", "email":"you@example.com" } } }</pre>

7.1.2. Network configuration parameters

You can customize your installation configuration based on the requirements of your existing network infrastructure. For example, you can expand the IP address block for the cluster network or configure different IP address blocks than the defaults.

- If you use the Red Hat OpenShift Networking OVN-Kubernetes network plugin, both IPv4 and IPv6 address families are supported.

- If you use the Red Hat OpenShift Networking OpenShift SDN network plugin, only the IPv4 address family is supported.

If you configure your cluster to use both IP address families, review the following requirements:

- Both IP families must use the same network interface for the default gateway.
- Both IP families must have the default gateway.
- You must specify IPv4 and IPv6 addresses in the same order for all network configuration parameters. For example, in the following configuration IPv4 addresses are listed before IPv6 addresses.

```
networking:
  clusterNetwork:
    - cidr: 10.128.0.0/14
      hostPrefix: 23
    - cidr: fd00:10:128::/56
      hostPrefix: 64
  serviceNetwork:
    - 172.30.0.0/16
    - fd00:172:16::/112
```



NOTE

Globalnet is not supported with Red Hat OpenShift Data Foundation disaster recovery solutions. For regional disaster recovery scenarios, ensure that you use a nonoverlapping range of private IP addresses for the cluster and service networks in each cluster.

Table 7.2. Network parameters

Parameter	Description	Values
<code>networking:</code>	The configuration for the cluster network.	Object  NOTE You cannot change parameters specified by the networking object after installation.
<code>networking: networkType:</code>	The Red Hat OpenShift Networking network plugin to install.	Either OpenShiftSDN or OVNKubernetes . OpenShiftSDN is a CNI plugin for all-Linux networks. OVNKubernetes is a CNI plugin for Linux networks and hybrid networks that contain both Linux and Windows servers. The default value is OVNKubernetes .

Parameter	Description	Values
networking: clusterNetwork:	The IP address blocks for pods. The default value is 10.128.0.0/14 with a host prefix of /23. If you specify multiple IP address blocks, the blocks must not overlap.	An array of objects. For example: <pre> networking: clusterNetwork: - cidr: 10.128.0.0/14 hostPrefix: 23 - cidr: fd01::/48 hostPrefix: 64 </pre>
networking: clusterNetwork: cidr:	Required if you use networking.clusterNetwork. An IP address block. If you use the OpenShift SDN network plugin, specify an IPv4 network. If you use the OVN-Kubernetes network plugin, you can specify IPv4 and IPv6 networks.	An IP address block in Classless Inter-Domain Routing (CIDR) notation. The prefix length for an IPv4 block is between 0 and 32. The prefix length for an IPv6 block is between 0 and 128. For example, 10.128.0.0/14 or fd01::/48.
networking: clusterNetwork: hostPrefix:	The subnet prefix length to assign to each individual node. For example, if hostPrefix is set to 23 then each node is assigned a /23 subnet out of the given cidr. A hostPrefix value of 23 provides 510 ($2^{(32 - 23)} - 2$) pod IP addresses.	A subnet prefix. For an IPv4 network the default value is 23. For an IPv6 network the default value is 64. The default value is also the minimum value for IPv6.
networking: serviceNetwork:	The IP address block for services. The default value is 172.30.0.0/16. The OpenShift SDN and OVN-Kubernetes network plugins support only a single IP address block for the service network. If you use the OVN-Kubernetes network plugin, you can specify an IP address block for both of the IPv4 and IPv6 address families.	An array with an IP address block in CIDR format. For example: <pre> networking: serviceNetwork: - 172.30.0.0/16 - fd02::/112 </pre>
networking: machineNetwork:	The IP address blocks for machines. If you specify multiple IP address blocks, the blocks must not overlap.	An array of objects. For example: <pre> networking: machineNetwork: - cidr: 10.0.0.0/16 </pre>

Parameter	Description	Values
networking: machineNetwork: cidr:	Required if you use networking.machineNetwork . An IP address block. The default value is 10.0.0.0/16 for all platforms other than libvirt and IBM Power® Virtual Server. For libvirt, the default value is 192.168.126.0/24 . For IBM Power® Virtual Server, the default value is 192.168.0.0/24 .	An IP network block in CIDR notation. For example, 10.0.0.0/16 or fd00::/48 .  NOTE Set the networking.machineNetwork to match the CIDR that the preferred NIC resides in.

7.1.3. Optional configuration parameters

Optional installation configuration parameters are described in the following table:

Table 7.3. Optional parameters

Parameter	Description	Values
additionalTrustBundle:	A PEM-encoded X.509 certificate bundle that is added to the nodes' trusted certificate store. This trust bundle might also be used when a proxy is configured.	String
capabilities:	Controls the installation of optional core cluster components. You can reduce the footprint of your OpenShift Container Platform cluster by disabling optional components. For more information, see the "Cluster capabilities" page in <i>Installing</i> .	String array
capabilities: baselineCapabilitySet:	Selects an initial set of optional capabilities to enable. Valid values are None , v4.11 , v4.12 and vCurrent . The default value is vCurrent .	String
capabilities: additionalEnabledCapabilities:	Extends the set of optional capabilities beyond what you specify in baselineCapabilitySet . You can specify multiple capabilities in this parameter.	String array

Parameter	Description	Values
cpuPartitioningMode:	Enables workload partitioning, which isolates OpenShift Container Platform services, cluster management workloads, and infrastructure pods to run on a reserved set of CPUs. You can only enable workload partitioning during installation. You cannot disable it after installation. While this field enables workload partitioning, it does not configure workloads to use specific CPUs. For more information, see the <i>Workload partitioning</i> page in the <i>Scalability and Performance</i> section.	None or AllNodes . None is the default value.
compute:	The configuration for the machines that comprise the compute nodes.	Array of MachinePool objects.
compute: architecture:	Determines the instruction set architecture of the machines in the pool. Currently, clusters with varied architectures are not supported. All pools must specify the same architecture. Valid values are amd64 and arm64 .	String
compute: hyperthreading:	Whether to enable or disable simultaneous multithreading, or hyperthreading , on compute machines. By default, simultaneous multithreading is enabled to increase the performance of your machines' cores.  IMPORTANT If you disable simultaneous multithreading, ensure that your capacity planning accounts for the dramatically decreased machine performance.	Enabled or Disabled
compute: name:	Required if you use compute . The name of the machine pool.	worker

Parameter	Description	Values
compute: platform:	Required if you use compute . Use this parameter to specify the cloud provider to host the worker machines. This parameter value must match the controlPlane.platform parameter value.	baremetal , vsphere , or {}
compute: replicas:	The number of compute machines, which are also known as worker machines, to provision.	A positive integer greater than or equal to 2 . The default value is 3 .
featureSet:	Enables the cluster for a feature set. A feature set is a collection of OpenShift Container Platform features that are not enabled by default. For more information about enabling a feature set during installation, see "Enabling features using feature gates".	String. The name of the feature set to enable, such as TechPreviewNoUpgrade .
controlPlane:	The configuration for the machines that form the control plane.	Array of MachinePool objects.
controlPlane: architecture:	Determines the instruction set architecture of the machines in the pool. Currently, clusters with varied architectures are not supported. All pools must specify the same architecture. Valid values are amd64 and arm64 .	String
controlPlane: hyperthreading:	Whether to enable or disable simultaneous multithreading, or hyperthreading, on control plane machines. By default, simultaneous multithreading is enabled to increase the performance of your machines' cores.  IMPORTANT If you disable simultaneous multithreading, ensure that your capacity planning accounts for the dramatically decreased machine performance.	Enabled or Disabled

Parameter	Description	Values
controlPlane: name:	Required if you use controlPlane . The name of the machine pool.	master
controlPlane: platform:	Required if you use controlPlane . Use this parameter to specify the cloud provider that hosts the control plane machines. This parameter value must match the compute.platform parameter value.	baremetal , vsphere , or {}
controlPlane: replicas:	The number of control plane machines to provision.	Supported values are 3 , or 1 when deploying single-node OpenShift.
credentialsMode:	The Cloud Credential Operator (CCO) mode. If no mode is specified, the CCO dynamically tries to determine the capabilities of the provided credentials, with a preference for mint mode on the platforms where multiple modes are supported.	Mint , Passthrough , Manual or an empty string (<code>""</code>). ^[1]
fips:	Enable or disable FIPS mode. The default is false (disabled). If you enable FIPS mode, the Red Hat Enterprise Linux CoreOS (RHCOS) machines that OpenShift Container Platform runs on bypass the default Kubernetes cryptography suite and use the cryptography modules that RHCOS provides instead.	false or true

Parameter	Description	IMPORTANT	Values
		To enable FIPS mode for your cluster, you must run the installation program from a Red Hat Enterprise Linux (RHEL) computer configured to operate in FIPS mode. For more information about configuring FIPS mode on RHEL, see Switching RHEL to FIPS mode. When running Red Hat Enterprise Linux (RHEL) or Red Hat Enterprise Linux CoreOS (RHCOS) booted in FIPS mode, OpenShift Container Platform core components use the RHEL cryptographic libraries that have been submitted to NIST for FIPS 140-2/140-3 Validation on only the x86_64, ppc64le, and s390x architectures. If you are using Azure File storage, you cannot enable FIPS mode.	
 imageContentSources:	Sources and repositories for the release-image content.		Array of objects. Includes a source and, optionally, mirrors , as described in the following rows of this table.

Parameter	Description	Values
imageContentSources: source:	Required if you use imageContentSources . Specify the repository that users refer to, for example, in image pull specifications.	String
imageContentSources: mirrors:	Specify one or more repositories that might also contain the same images.	Array of strings
publish:	How to publish or expose the user-facing endpoints of your cluster, such as the Kubernetes API, OpenShift routes.	Internal or External . The default value is External . Setting this field to Internal is not supported on non-cloud platforms.
sshKey:	The SSH key to authenticate access to your cluster machines.  NOTE For production OpenShift Container Platform clusters on which you want to perform installation debugging or disaster recovery, specify an SSH key that your ssh-agent process uses.	For example, sshKey: ssh-ed25519 AAAA...

1. Not all CCO modes are supported for all cloud providers. For more information about CCO modes, see the "Managing cloud provider credentials" entry in the *Authentication and authorization* content.

7.2. AVAILABLE AGENT CONFIGURATION PARAMETERS

The following tables specify the required and optional Agent configuration parameters that you can set as part of the Agent-based installation process.

These values are specified in the **agent-config.yaml** file.



NOTE

These settings are used for installation only, and cannot be modified after installation.

7.2.1. Required configuration parameters

Required Agent configuration parameters are described in the following table:

Table 7.4. Required parameters

Parameter	Description	Values
apiVersion:	The API version for the agent-config.yaml content. The current version is v1beta1 . The installation program might also support older API versions.	String
metadata:	Kubernetes resource ObjectMeta , from which only the name parameter is consumed.	Object
metadata: name:	The name of the cluster. DNS records for the cluster are all subdomains of {{.metadata.name}} . {{.baseDomain}} . The value entered in the agent-config.yaml file is ignored, and instead the value specified in the install-config.yaml file is used. When you do not provide metadata.name through either the install-config.yaml or agent-config.yaml files, for example when you use only ZTP manifests, the cluster name is set to agent-cluster .	String of lowercase letters and hyphens (-), such as dev .

7.2.2. Optional configuration parameters

Optional Agent configuration parameters are described in the following table:

Table 7.5. Optional parameters

Parameter	Description	Values
rendezvousIP:	The IP address of the node that performs the bootstrapping process as well as running the assisted-service component. You must provide the rendezvous IP address when you do not specify at least one host's IP address in the networkConfig parameter. If this address is not provided, one IP address is selected from the provided hosts' networkConfig .	IPv4 or IPv6 address.

Parameter	Description	Values
bootArtifactsBaseURL:	The URL of the server to upload Preboot Execution Environment (PXE) assets to when using the Agent-based Installer to generate an iPXE script. For more information, see "Preparing PXE assets for OpenShift Container Platform".	String.
additionalNTP Sources:	A list of Network Time Protocol (NTP) sources to be added to all cluster hosts, which are added to any NTP sources that are configured through other means.	List of hostnames or IP addresses.
hosts:	Host configuration. An optional list of hosts. The number of hosts defined must not exceed the total number of hosts defined in the install-config.yaml file, which is the sum of the values of the compute.replicas and controlPlane.replicas parameters.	An array of host configuration objects.
hosts: hostname:	Hostname. Overrides the hostname obtained from either the Dynamic Host Configuration Protocol (DHCP) or a reverse DNS lookup. Each host must have a unique hostname supplied by one of these methods, although configuring a hostname through this parameter is optional.	String.
hosts: interfaces:	Provides a table of the name and MAC address mappings for the interfaces on the host. If a NetworkConfig section is provided in the agent-config.yaml file, this table must be included and the values must match the mappings provided in the NetworkConfig section.	An array of host configuration objects.
hosts: interfaces: name:	The name of an interface on the host.	String.
hosts: interfaces: macAddress:	The MAC address of an interface on the host.	A MAC address such as the following example: 00-B0-D0-63-C2-26 .

Parameter	Description	Values
<code>hosts: role:</code>	Defines whether the host is a master or worker node. If no role is defined in the agent-config.yaml file, roles will be assigned at random during cluster installation.	master or worker .
<code>hosts: rootDeviceHints:</code>	Enables provisioning of the Red Hat Enterprise Linux CoreOS (RHCOS) image to a particular device. The installation program examines the devices in the order it discovers them, and compares the discovered values with the hint values. It uses the first discovered device that matches the hint value. This is the device that the operating system is written on during installation.	A dictionary of key-value pairs. For more information, see "Root device hints" in the "Setting up the environment for an OpenShift installation" page.
<code>hosts: rootDeviceHints: deviceName:</code>	The name of the device the RHCOS image is provisioned to.	String.
<code>hosts: networkConfig:</code>	The host network definition. The configuration must match the Host Network Management API defined in the nmstate documentation .	A dictionary of host network configuration objects.

Additional resources

- [Preparing PXE assets for OpenShift Container Platform](#)
- [Root device hints](#)